THE STACK, THE CODE, THE TECHNIQUES

CareCall Inside Out

How an AI voice agent for elderly care is built: every technology, every file, and the techniques behind them

Python, FastAPI and asyncio; WebSockets, Twilio and streaming speech; LangGraph and tool-calling agents; PostgreSQL, idempotency and locks; Langfuse and evaluation

Edition 2
19 October 2026

Contents

Preface 2

PART I. ORIENTATION 4

1. The Product and the System on One Page 5
2. Every Technology, in One Table 8

PART II. THE STACK 10

3. Python Runtime: HTTP, FastAPI, asyncio and Pydantic 11
4. Real-Time Audio: WebSockets, Twilio and Speech 14
5. Language Models 17
6. LangGraph and the Conversational Agent 20
7. Data, State and Reliability Concepts 24
8. Tracing, Testing and Infrastructure 28

PART III. THE CODE, FILE BY FILE 30

9. Foundations and Entry Points 31
10. The agent Package 35
11. The calls Package 39
12. The telephony Package 42
13. Model Clients, Prompts and Personas 44
14. The evals Package 47
15. The Tests 50
16. Infrastructure and Documentation 52

PART IV. TECHNIQUES IN DEPTH 54

17. How the Agent Gets Its Context 55
18. How Speech Is Streamed to the Phone 59
19. Tool Calling Without a Tool-Calling API 63
20. Reliability: Keys, Savepoints and Locks 68
21. Testing Behaviour That Varies 72
22. Tracing One Call Across Threads 76

PART V. CHOICES AND ALTERNATIVES 79

23. Alternative Stacks and Their Tradeoffs 80

PART VI. OPERATING IT 94

24. Running It, Measured Results and What Is Missing 95

Appendix A. Glossary 99

BEFORE YOU BEGIN

Preface

This book explains how CareCall is built: every technology it uses, every file in the repository, and the handful of techniques that are not obvious from reading the code line by line.

CareCall is an AI agent that phones elderly parents on a schedule. It reminds them about medicines and appointments, asks what they ate and records it, books a call-back or a new reminder when they ask, and tells their adult children (the caregivers) what happened. The project was built to learn, so the book spends its words on *why* each piece exists and what the alternatives would have cost.

How the book is organised

- **Part I, Orientation.** The product, one picture of the whole system, and the life of a single call. Then a table of every technology and where it lives.
- **Part II, The Stack.** Each technology explained from first principles, then how this project uses it, then the tradeoff: Python's concurrency model, WebSockets and phone audio, language models, LangGraph, PostgreSQL, tracing and tests.
- **Part III, The Code, file by file.** Every source file, what it contains and how it connects to its neighbours at runtime. Plain syntax is only touched on; anything unusual gets a pointer to Part IV.
- **Part IV, Techniques in depth.** The things worth understanding properly: how the agent keeps the context of a conversation, how speech is streamed to the phone, how tools are called without a tool-calling API, how duplicate and racing calls are prevented, how non-deterministic behaviour is tested, and how one call is traced across threads.
- **Part V, Choices and alternatives.** The other stacks you could have used, including live speech-to-speech models, and a checklist for your own next application.
- **Part VI, Operating it.** Commands, settings, measured results and an honest list of what is still missing. A glossary follows.

One idea that explains most of the design

Guarantees live in code; judgment lives in the model. A language model is excellent at understanding messy human speech and unreliable as the last line of defence. So the model decides what to say and which tool to use, but everything that must always hold is enforced by ordinary code: the quiet hours, the cap on attempts, the rule that a logged fact must quote the parent's real words, the emergency check, the limit on repeated lines. When you wonder why a piece of code exists, ask which guarantee it protects.

The state of the system

The book describes the system as of 19 October 2026. The suite of 230 automated tests passes. The conversational agent has been exercised against simulated parents played by a language model, and a tool-selection test checks that it calls the right tools: 111 to 113 of 116 runs were correct on the cheap model and 114 of 116 on a stronger one. Streaming speech has been measured against the real speech service. A real outbound Twilio call has not yet been placed, and several production concerns (database migrations, an onboarding screen, secrets management) are still open. Chapter 24 lists them.

PART I

Orientation

What CareCall does, how the pieces fit together, and every technology in one table.

CHAPTER 1

The Product and the System on One Page

CareCall phones a parent at the right time, holds a short conversation, writes down what it learned, and decides what happens next. Everything in the repository serves one of three loops that run at different speeds.

Loop	Speed	What it does	Main files
Schedule loop	seconds to hours	Turns stored routines into dated calls, dials the ones that are due, and schedules the follow-up after each call ends	`calls/planner.py`, `scheduler.py`, `outcome.py`
Call loop	minutes	One phone call: audio in, text, agent, text, audio out	`telephony/bridge.py`, `agent/session.py`
Turn loop	about a second	One parent utterance: the agent decides what to say and which tools to run	`agent/converse.py`, `agent/db_tools.py`

The system on one page

The server is one Python process. It exposes HTTP endpoints for Twilio's webhooks and an admin view, a WebSocket endpoint for live call audio, and can run the scheduler loop in the background. It calls out to four external services: Gnani for speech, a language model through OpenRouter, PostgreSQL for everything it remembers, and Langfuse for traces.

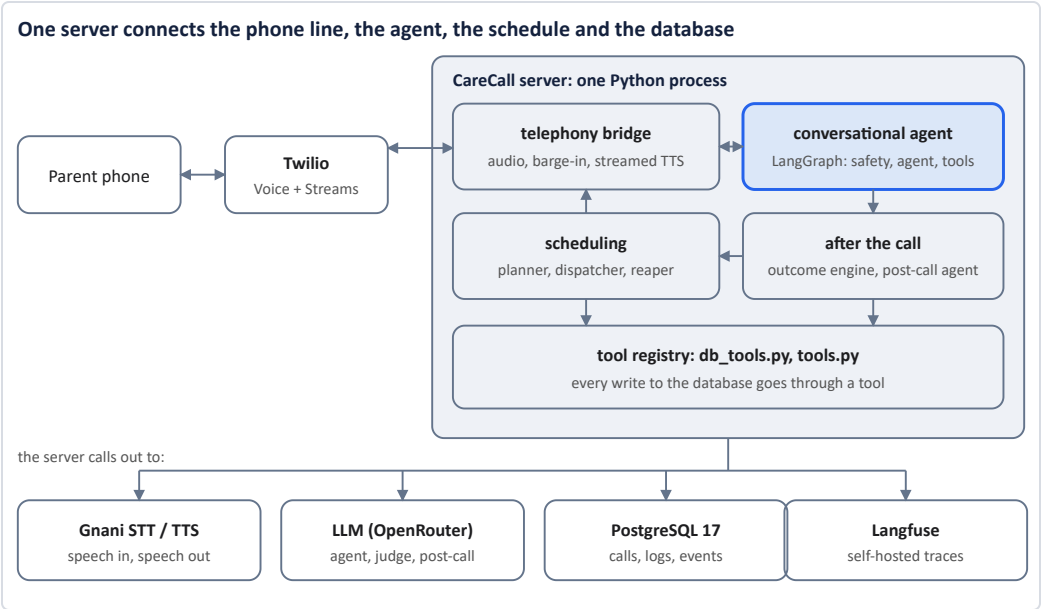


Figure 1. *Architecture at a glance. Twilio talks to the telephony bridge. The bridge feeds the conversational agent. The scheduler dials calls (up to the bridge); the agent hands each finished call to the after-call engine, which books the next call back into the scheduler. Every write goes through the tool registry into PostgreSQL.*

The life of one call

1. **Onboarding data exists.** A parent row, a caregiver, meal times, medicines, reminders, and routines such as `lunch_checkin`. Today `seed.py` creates them; there is no onboarding screen yet.
2. **The planner turns that data into a calendar.** `calls/planner.py` runs hourly and decides, in the parent's own timezone, when each routine fires. Each becomes a `call_attempts` row with status `scheduled` and a computed idempotency key.
3. **The dispatcher claims what is due.** `calls/scheduler.py` selects due rows with `FOR UPDATE SKIP LOCKED`, checks guards (parent already on a call, quiet hours, calling switched off), marks the row `dialing`, commits, and only then asks Twilio to ring.
4. **Twilio rings and the parent answers.** Twilio asks our `/twilio/voice` webhook what to do and receives XML that opens a media stream to our WebSocket.
5. **The bridge starts a session.** `telephony/bridge.py` verifies a per-call token, builds a `CallSession`, connects to Gnani speech-to-text, and speaks the opening line.

6. **Each parent utterance is one turn.** Final transcripts from the speech service are passed to `CallSession.turn`, which runs the conversational graph: the model is shown the whole conversation, answers with JSON, and any tools it asked for are run.
7. **The reply is spoken while it is still being generated.** Text goes to the streaming text-to-speech endpoint, and audio frames flow back to the phone as they arrive.
8. **The call ends.** The bridge notes whether the parent was mid-speech, `CallSession.finish` writes a `call.finished` event, and the post-call pipeline writes the summary and notifies the family.
9. **The outcome engine decides the next step.** Completed, cut off, hung up on purpose, never connected, call-back requested, emergency or opt-out. Follow-ups are scheduled with keys that match the original call.
10. **Tomorrow is planned.** A finished routine schedules its next day, and the loop continues.

CHAPTER 2

Every Technology, in One Table

Every technology in the project, in one table. Chapters 3 to 8 explain each; Chapters 9 to 16 show where it appears in the files.

Layer	Technology	What it does here	Main files
Language	Python 3.12	The whole backend; `async` / `await` for concurrency	everything
Web framework	FastAPI on ASGI, served by uvicorn	Twilio webhooks, admin endpoints, the audio WebSocket	`api.py`
Validation and settings	Pydantic v2, pydantic-settings	Typed models for settings, call context and model output	`config.py`, `agent/context.py`, `agent/interpret.py`
Concurrency	asyncio, worker threads, process pool	Many calls on one event loop; blocking work in threads; parallel evals in processes	`telephony/bridge.py`, `evals/*`
HTTP client	httpx (sync and async), forced IPv4	Calls to the model, Gnani and Twilio	`net.py`, `llm/openai_compat.py`
Realtime transport	WebSocket (Starlette and the `websockets` library), SSE	Twilio audio in and out, streaming speech-to-text, streaming text-to-speech	`telephony/*`
Telephony	Twilio Voice, TwiML, Media Streams	Place the call, carry its audio	`telephony/twilio_api.py`, `bridge.py`
Audio	mu-law and PCM conversion, resampling, 20 ms framing (`audioop`)	Convert between phone audio and what the speech services use	`telephony/audio.py`

Layer	Technology	What it does here	Main files
Speech	Gnani Prisma (STT) and Timbre (TTS)	Speech to text and text to speech, Indian languages	`telephony/gnani_speech.py`
Language models	OpenAI-compatible API through OpenRouter	The conversation, the post-call decision, judges and simulated parents	`llm/*`, `agent/converse.py`
Orchestration	LangGraph with an in-memory checkpointer	The per-call graph: nodes, routing, saved state per call	`agent/converse.py`, `agent/graph.py`
Database	PostgreSQL 17, SQLAlchemy 2, psycopg 3	Calls, transcripts, logs, events, queue	`models.py`, `db.py`, `calls/*`
Reliability patterns	idempotency keys, SAVEPOINT, `FOR UPDATE SKIP LOCKED`, state machine	No duplicate calls, safe with many workers	`calls/keys.py`, `service.py`, `scheduler.py`, `state.py`
Observability	Langfuse (self-hosted) on OpenTelemetry	Trace each call, each turn, each tool and each model call	`tracing.py`
Evaluation	pytest, persona simulations, LLM judges, tool-selection test	Check behaviour that is not deterministic	`evals/*`, `tests/*`
Packaging	Docker Compose, `pyproject.toml`, a `carecall` command	Run the databases; install the package and CLI	`docker/*`, `cli.py`
Notifications	console notifier, optional Twilio SMS	Tell the family	`calls/notify.py`

PART II

The Stack

Each technology from first principles: what it is, how this project uses it, and the tradeoff.

CHAPTER 3

Python Runtime: HTTP, FastAPI, asyncio and Pydantic

This chapter covers the plumbing every other chapter relies on: how a Python server handles many phone calls at once, and how the data crossing its edges is checked.

HTTP, REST and webhooks

HTTP is request and response: a client sends a request, the server sends one response, and the exchange is over. **REST** is a convention on top of it: URLs name things (`/calls/12`) and methods say what to do (`GET` reads, `POST` creates). A **webhook** is the same thing reversed: instead of us calling a service, the service calls us when something happens.

CareCall uses plain HTTP for everything that is not live audio. Twilio calls `POST /twilio/voice` to ask "the parent answered, what should I do?" and `POST /twilio/status` to report ringing, answered, completed, busy or no-answer. The admin endpoints (`/admin/calls` , `/admin/parents` , `PUT /admin/parents/{id}/meal-times`) are ordinary REST. The tradeoff is that HTTP cannot carry a continuous two-way stream with low delay, which is why audio uses WebSockets (Chapter 4).

FastAPI, ASGI and uvicorn

FastAPI is the web framework. You write a function, decorate it with `@app.post("/calls")` , and the framework parses the request, validates it against a Pydantic model and serialises the response. It sits on **ASGI**, the async interface between a Python web application and its server. **uvicorn** is the server: it accepts network connections and hands each HTTP request or WebSocket to the application.

The reason for ASGI is that it supports WebSockets and `async` natively. The older WSGI standard (Flask, Django) runs one request per thread and has no first-class WebSocket story, which a phone-call server needs. `api.py` builds the app in

`create_app(...)`, which accepts factories for the speech services and the interpreter so tests can plug in fakes.

One detail worth knowing: FastAPI's **lifespan** hook runs code on startup and shutdown. CareCall uses it to initialise the database, seed demo data, start tracing and, when `ENABLE_SCHEDULER` is true, start the scheduler loop as a background task that is stopped cleanly on shutdown.

Pydantic: data that is checked at the door

Pydantic defines typed data models with validation. A model that returns `"reason": null` or a wrong type is rejected or normalised at the boundary instead of crashing deep inside the code. CareCall uses it in three places:

- `config.Settings` (via pydantic-settings): every setting, read from environment variables and the git-ignored `.env`, with defaults. `get_settings()` is cached with `lru_cache`.
- `agent.context.CallContext`: everything the agent is allowed to know about one call, built once when the call opens and stored as an audit event.
- `agent.interpret.Interpretation`: the structured answer the older, scripted agent asked the model for. A `model_validator` drops nulls so a model's `null` means "not provided".

asyncio: many calls on one thread

A phone call is mostly waiting: for the parent to speak, for the speech service, for the model, for audio to play. **asyncio** runs many such waiting tasks on one thread using an *event loop*. An `async def` function can `await` something slow; while it waits, the loop runs another call's code. The pieces CareCall uses:

Piece	What it is	Where it appears
Task (`asyncio.create_task`)	A coroutine running in the background	The bridge runs one task pumping speech events, one consuming turns, one watching for silence
`asyncio.Queue`	A thread-of-control-safe list one task fills and another empties	Speech-to-text events go into `inbox`; `_consume` takes them out
`asyncio.Event`	A flag a task can wait on	`_mark_done` is set when Twilio reports it finished playing our audio

Piece	What it is	Where it appears
`asyncio.to_thread`	Run a blocking function in a worker thread and `await` it	The agent turn and all database work; the event loop stays free for other calls
`run_coroutine_threadsafe`	Let a *thread* schedule a coroutine back onto the loop	The filler "Umm, hold on" is spoken from the graph's worker thread while a tool runs

The tradeoff is discipline. One blocking call on the event loop freezes every call, so everything synchronous (the LangGraph invoke, SQLAlchemy sessions, the sync httpx calls to the model) is wrapped in `to_thread`. The default thread pool is finite, so past some number of simultaneous turns they queue; that limit has not been load-tested.

Threads versus processes

The conversational graph is synchronous and may take a second or two, so each turn runs in a worker thread. Evaluations are different: each simulated call uses its own in-memory database, and SQLAlchemy's engine is a per-process global. So the persona and tool-selection runs use a `ProcessPoolExecutor`, one process per case, which gives isolation for free.

httpx and the IPv4 workaround

All outbound HTTP goes through `net.py`, a tiny factory that returns an httpx client bound to `0.0.0.0` so only IPv4 is used. The reason is practical: on some networks IPv6 is advertised in DNS but does not work, so every new connection waited for each IPv6 address to time out (40 to 60 seconds) before falling back. The WebSocket client for speech-to-text applies the same fix with `family=socket.AF_INET`. `FORCE_IPV4=false` turns it off.

CHAPTER 4

Real-Time Audio: WebSockets, Twilio and Speech

This chapter follows a parent's voice from the phone network to text and back again. Nothing here involves a language model; it is all transport and signal handling, and getting it wrong sounds like static, delay or two people talking over each other.

WebSocket

A **WebSocket** starts as an HTTP request that is *upgraded* into a long-lived two-way channel. Either side can send a message at any moment, with tiny headers and no new connection per message. Two WebSockets are in play on every call:

1. **Twilio to us** at `/twilio/stream`. Twilio sends JSON messages (`start`, `media`, `mark`, `stop`) and we send back `media`, `mark` and `clear`.
2. **Us to Gnani speech-to-text** at `wss://api.vachana.ai/stt/v3/stream`. We send raw audio and receive transcripts.

Server-Sent Events (SSE) is a simpler, one-way cousin: a normal HTTP response that never ends, with a stream of `event:` and `data:` lines. CareCall uses it for streaming text-to-speech (Chapter 18).

Twilio Voice, TwiML and Media Streams

Twilio is the telephone network. Placing a call is a REST request to Twilio (`calls.create`) that includes two URLs of ours: one for instructions and one for status updates. When the parent answers, Twilio fetches **TwiML**, an XML instruction document, from our `/twilio/voice` endpoint. Ours says "connect this call's audio to a WebSocket":

```
<Response><Connect><Stream url="wss://.../twilio/stream">
  <Parameter name="call_id" value="12"/><Parameter name="token" value="..."/>
```

```
</Stream></Connect><Hangup/></Response>
```

`telephony/twilio_api.py` builds this with Twilio's helper classes. Two security measures protect it. Twilio's webhooks carry an `X-Twilio-Signature` header that `validate_signature` checks. And since Twilio cannot send auth headers on a media stream, we put a per-call HMAC token in the stream parameters (`make_token(call_id)` signs the string `call:<id>` with `STREAM_SECRET`), and the bridge refuses any stream whose token does not verify.

What the audio looks like

Telephone audio is **mu-law, 8 kHz, mono**, delivered in 20 ms frames of 160 bytes, base64-encoded inside the JSON `media` messages. Speech services want ordinary 16-bit PCM. `telephony/audio.py` converts between them:

Function	Job
`mulaw_to_pcm16`, `pcm16_to_mulaw`	Decode and encode G.711 using the standard-library `audioop` (or `audioop-lts` on Python 3.13)
`resample_pcm16(data, from, to, state)`	Change sample rate. It returns a `state` that must be passed back for the next chunk, so chunk boundaries do not click
`frames(mulaw)`	Cut audio into exact 160-byte frames, padding the last frame with silence (`0xFF`)

Because `frames` pads, the size of every chunk you hand it matters. Chapter 18 explains why this shaped the streaming code.

Speech to text (Gnani Prisma)

`GnaniSTT` in `telephony/gnani_speech.py` opens the WebSocket with its settings in the upgrade *headers* (language, sample rate, voice-activity thresholds). The server has three peculiarities the adapter respects:

- Audio must be binary frames of **exactly 1024 bytes** (512 samples). `send_audio` keeps a `bytearray` buffer and slices exactly 1024 bytes at a time, however large the input chunk is.
- The server does its own **voice activity detection** and sends one `transcript` message per speech segment, plus a `speech_start` message when someone begins.
- Audio is expected at roughly real-time pace, which the phone provides for free.

The adapter turns server messages into `SttEvent` objects on an `asyncio.Queue`. The bridge uses `speech_start` events to know the parent is mid-speech, which matters when a call drops (Chapter 12).

Text to speech (Gnani Timbre)

`GnaniTTS` turns a line of text into raw 16-bit PCM at 8 kHz. There are two endpoints: REST (`/tts/inference`) returns the whole clip in one response; SSE (`/tts/sse`) streams the clip as it is generated. The adapter yields exact 20 ms chunks either way, so the bridge does not care which one is used. A setting, `TTS_STREAMING` (default on), chooses, and streaming falls back to REST if it fails before any audio. Chapter 18 explains the technique and the measured gain.

Barge-in, marks and silence

People interrupt. When a *partial* transcript (non-final, non-empty) arrives while we are still playing audio, the bridge sends Twilio a `clear` message, which flushes everything queued for playback so we stop talking at once. To know when playback has *finished*, the bridge sends a `mark` message after the audio; Twilio echoes it back when it reaches that point in the stream, and the bridge sets `playing = False` and starts a **silence timer** (seven seconds by default). If the timer fires with no speech, the bridge feeds the agent an empty utterance, which the agent treats as "(silence: the person said nothing)".

Tradeoffs

- **Narrowband audio.** Phone audio at 8 kHz loses detail compared with the wideband audio of an app or WebRTC. It is what the phone network gives us.
- **Cascade versus speech-to-speech.** CareCall chains speech-to-text, a language model and text-to-speech. A single speech-to-speech model would cut the delay and handle interruption natively, but every stage here produces text that can be logged, checked and tested. Chapter 23 compares the options.

CHAPTER 5

Language Models

How the project talks to a model

Models are reached through an **OpenAI-compatible HTTP API**, pointed at OpenRouter. The consequence is that the model is a *setting*, not a code change: `LLM_BASE_URL`, `LLM_API_KEY` and `LLM_MODEL` in `.env` name an endpoint and a model, and swapping from one vendor's model to another is editing a string. `llm/openai_compat.py` is the entire client. It posts to `/chat/completions`, merges an `extra_body` dictionary into every request (the place for provider-specific options), retries on timeouts, 429s and 5xx errors, and returns the message text.

Four model roles are configured separately in `llm/factory.py`, because each has a different need:

Role	Factory	Needs	Used by
Conversation agent	`make_agent_llm_client()`	Fast and obedient; runs while a parent waits	`agent/converse.py`
Judge and post-call decision	`make_judge_client()`	Stronger; runs off the live path	`calls/postcall_agent.py`, `evals/*`
Simulated parent	`make_persona_client()`	Livelier (higher temperature)	`evals/personas.py`
Interpreter (older agent)	`make_llm_client()`	Cheap, JSON-reliable	`agent/graph.py`

`AGENT_LLM_*` variables override the shared `LLM_*` for the conversation agent, which is how a stronger model can be tried without touching code.

Fast replies: no thinking, JSON mode, small outputs

Three choices keep a turn near one second. **Thinking is disabled** through `extra_body` (`{"reasoning": {"enabled": false}}`), so the model answers immediately instead of reasoning first. **JSON mode** (`response_format: json_object`, in `chat_json`) makes the model return a single JSON object, which

removes a class of parsing problems. And the **reply is short by design**: a typical model call has about 2,800 input tokens (mostly the instructions) and under 60 output tokens.

The client also counts its own calls, seconds and tokens in a `stats` dictionary. The tool-selection test reads those numbers to compare models on latency and cost.

Why the model returns JSON actions instead of using function calling

Most chat APIs offer native *function calling*: you describe tools and the model emits a structured call. CareCall does not rely on it. Each model step returns one JSON object `{"say": ..., "actions": [{"tool": ..., "args": ...}], "end_call": ...}`, and our own code parses and runs the actions. The reasons:

- It works with any model, including ones whose tool-calling support is weak.
- One round trip usually suffices: the model can speak *and* log a fact in the same reply, where native function calling needs a second call to continue after a tool result.
- The validation pipeline (Chapter 19) is ours, so the rules do not depend on a vendor's behaviour.

The cost is that malformed JSON must be handled, which `_llm_step` does with one retry and then a polite fallback.

Prompts

The conversation agent's instructions are built per call by `system_prompt()` in `agent/converse.py` from the call context, so a medicine reminder, a meal check-in and a reminder call each get a task description and rules for their own situation. A few principles are worth noting because they came from testing:

- **Strict scope.** The agent does three things (remind, log diet, schedule). Everything else is declined in one sentence and the question is asked again.
- **Shapes of replies.** The prompt shows the *shape* of each kind of reply (decline, medical referral, loneliness) because a small model tends to paste the wrong sentence into the wrong situation unless shown which sentence belongs where.
- **Tools generated from the registry.** The tool list in the prompt is produced by `tools_text()` from the same registry that executes the tools, so the prompt and the code cannot drift apart.

Older, smaller prompts live in `prompts/` and serve the scripted interpreter; they are versioned by a short hash so any call can be tied to the exact text that produced it.

Cheap model or stronger model?

The same 58-case tool-selection test was run on both. The cheap non-thinking model scored 111 to 113 of 116; the stronger one scored 114 of 116 and fixed three specific weaknesses (not booking a time on its own after a refusal, raising an alert for a medical question, and logging a vague answer instead of looping) but made one new mistake and was about 0.3 seconds slower and roughly three times the price per call. Prices come from the provider's listing, and measured results are in Chapter 24.

CHAPTER 6

LangGraph and the Conversational Agent

What LangGraph is

LangGraph is a library for building stateful programs as a graph. You define a typed **state** (a dictionary every step can read and update), **nodes** (ordinary functions that take the state and return a *partial update*), and **edges** (which node runs next). A **conditional edge** calls a small routing function that inspects the state and picks the next node, so the program can branch and loop. A **checkpointer** saves the state after each step under a `thread_id`, so the next call on the same thread continues where the last one stopped.

You compile the graph and call `graph.invoke(input, config)`. Each call runs from the start node, follows edges until it reaches `END`, and returns the final state. What the library adds over plain `if` statements is an explicit, inspectable flow, per-node tracing, and state that survives between calls.

The conversational graph

In CareCall one `invoke` handles one parent utterance (plus one at the start to greet). The graph is built by `build_conversation_graph(llm, speak_now)` in `agent/converse.py`. Its shape is the classic agent loop: ask the model, run what it asked for, show it the results, repeat until it has something to say.

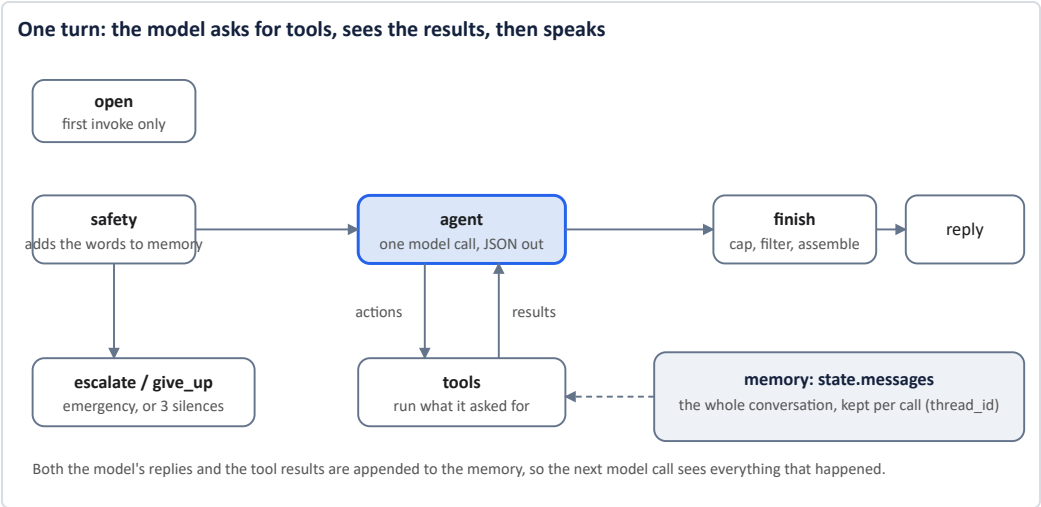


Figure 2. *The graph for one turn. If the model asks for a tool whose answer matters (a booking, a lookup) or a tool fails, the loop goes agent, tools, agent again, up to four steps, before the turn ends in* `finish`.

The state

The state is a `TypedDict` called `ConvState`. The fields that matter most:

Field	Meaning
`ctx`	The call context (parent, caregiver, routine, medicine, local time), as a dictionary
`system`	The system prompt, built once when the call opens
`messages`	The conversation so far, as a list of `{"role", "content"}` dictionaries. This is the agent's memory
`utterance`	What the parent just said (the new input for this invoke)
`pending`, `say`, `end_call`	The model's latest request: tool calls to run, words to speak, whether to end
`iterations`, `again`	How many model steps this turn, and whether the tools node wants another
`fillers`, `early`, `filler_n`	Stalling lines ("Umm, hold on") for this turn, which of them were already spoken live, and a counter so the wording rotates
`silence_run`, `parent_turns`	Consecutive silent turns, and total turns (for the call limits)
`booked`, `emergency`	Sticky flags that decide how the call is classified when it ends
`reply`, `done`, `end_reason`	The output of the turn

LangGraph keeps this state for the whole call. Each node returns only the keys it changed, and LangGraph merges them in (a later value replaces an earlier one). A few keys are reset at the start of each turn by the `safety` node (`pending`, `say`, `iterations`, `tool_log`, `fillers`, `early`) so one turn's leftovers never leak into the next.

The nodes

- **`open`**: runs on the first invoke only. It builds the system prompt, produces the opening (a fixed greeting plus the routine's exact first question, so medicine wording is never improvised), and seeds `messages`. For a ring-back after a cut-off call it asks the model for the continuation instead, with the earlier conversation included in the prompt.
- **`safety`**: appends the parent's words to `messages`, resets per-turn fields, and runs the emergency regex. It never calls a model.
- **`escalate`** and **`give_up`**: end the turn without the model. `escalate` raises an emergency alert and says the family is being told. `give_up` handles three silent turns in a row and still leaves a record (an acknowledgement of "no" for a reminder, an "unclear" medicine log, an "unclear" meal log).
- **`agent`**: one model call through `_llm_step`. It stores the model's own reply in `messages` and puts its requested actions in `pending`.
- **`tools`** (`run_tools`): executes each action through the tool registry, speaks a filler if a slow tool is running, and appends a `TOOL RESULTS:` message when the model must see the outcome.
- **`finish`**: caps the reply length, filters leaks, applies the repeated-line guard, assembles the reply list, and decides whether the call ends and why.

Routing

Three small functions choose the next node. `route_start` sends the first invoke to `open` and every later one to `safety`. After `safety`, an emergency goes to `escalate`, three silences go to `give_up`, anything else goes to `agent`. After `agent`, pending actions go to `tools`, otherwise to `finish`. After `tools`, `again` goes back to `agent`, otherwise to `finish`.

Checkpointing and the thread id

The graph is compiled with `MemorySaver`, an in-process checkpointer, and each call uses `thread_id = call-<id>`. That is why `CallSession.turn` can pass only the new utterance: LangGraph restores the rest of the state for that thread. The tradeoff is that the memory lives in the server's process; if the process dies mid-call, the call cannot resume from the checkpoint. That is acceptable because a dropped phone call cannot be resumed anyway, and what matters is written to PostgreSQL as it happens.

The speak-now hook

`build_conversation_graph` receives `speak_now`, a function that *returns* a callback (or `None`). On a phone call the bridge supplies one so a filler can be spoken the instant a tool starts, while the graph's worker thread continues. In text sessions there is no callback and the filler simply leads the reply. Chapter 18 explains the mechanism.

The older, scripted graph

`agent/graph.py` is the first agent: a fixed list of questions, a model that fills a form (`Interpretation`) for each answer, and code that validates the form and decides what to say. It is kept under `mode="graph"` for comparison and because most of the older tests exercise it. A session picks the graph by an explicit `mode`, or by whether an interpreter was passed (an interpreter means the scripted graph), otherwise by the `AGENT_MODE` setting, which defaults to the conversational agent.

CHAPTER 7

Data, State and Reliability Concepts

PostgreSQL, SQLAlchemy and psycopg

The database is **PostgreSQL 17**, run in Docker on port 5433. **SQLAlchemy 2** is the object-relational mapper: each table is a Python class in `models.py`, and `session_scope()` in `db.py` gives a transaction that commits on success and rolls back on error. **psycopg 3** is the driver. The engine uses a connection pool (10 connections plus 20 overflow), `pool_pre_ping` so dead connections are dropped, and recycling every 30 minutes.

PostgreSQL was chosen over SQLite because SQLite allows one writer at a time and has no row-level locking, so it cannot safely serve several calls and several worker processes at once. PostgreSQL provides real concurrency, `SELECT ... FOR UPDATE SKIP LOCKED`, and savepoints. SQLite is kept only for tests; it returns timezone-naive datetimes, which is why `models.as_utc()` exists.

There are no migrations yet. Tables are created with `Base.metadata.create_all`. Alembic is the next step before any production data exists.

The data model

The schema has 18 tables, grouped by purpose.

Group	Tables	Holds
People	`parents`, `caregivers`, `parent_caregiver`	Who is called and who is told; timezone; the `calling_enabled` switch
What to ask about	`meal_profile`, `medicines`, `reminders`, `routines`	Meal times (with a source), medicines, one-off reminders, and routines linking a type to its config
Calls	`call_attempts`, `transcript_turns`	Every planned or made call with status, keys, outcome; each spoken line with latency
Facts learned	`meal_logs`, `medicine_logs`, `reminder_acks`	What the parent said, each with their own words as evidence

Group	Tables	Holds
Promises and alerts	`callbacks`, `alerts`	Call-backs promised; things the family must know, with severity
Outputs	`call_summaries`, `notifications`	The caregiver-facing summary and the outbox of messages sent
Audit	`events`, `test_notes`	An append-only log of everything that happened; notes written by the evaluation tools

The call state machine

A call is in exactly one state, and `calls/state.py` allows only legal moves:

From	To
scheduled	dialing, cancelled (or in_progress directly on the text channel)
dialing	ringing, in_progress, no_answer, failed
ringing	in_progress, no_answer, failed
in_progress	completed, abandoned, failed
completed, abandoned, no_answer, failed, cancelled	final

A state machine makes webhooks safe. Twilio may report the same status twice or out of order; a repeated or illegal transition is ignored (and recorded as a `state.ignored` event) instead of corrupting the call.

Idempotency, in one paragraph

An operation is **idempotent** if doing it twice has the same effect as doing it once. CareCall needs that wherever a retry, a duplicate webhook, two workers or a re-run could create a second call. The tool is a **unique idempotency key** on each call, computed rather than random: `parent:routine:local-date:slot` for a planned call (the *occurrence key*), then `<occurrence>:a<n>` for a retry and `<occurrence>:cb<n>` for a call-back. Inserting a call whose key already exists returns the existing row. This is separate from the **active-call guard**, which stops *different* calls ringing the same parent at once. Chapter 20 shows the code.

The outcome engine and the retry policy

When a call ends, `calls/outcome.py` classifies it as one of `completed`, `callback_requested`, `cut_off`, `hung_up_on_purpose`, `never_connected`, `technical_failure`, `emergency`, `off_topic_limit`, `opted_out` or `cancelled`, and schedules the follow-up. The policy numbers live in `config.py`:

- At most **2 attempts per reminder**: the first call plus one more, then the family is told and the system moves on.
- A retry that would land after the **next reminder** is dropped.
- A call that dropped while the parent was mid-sentence is **rung back immediately** and resumes where it stopped.
- No answer: retry once after 10 minutes. Technical failure: once after 5. Hung up on purpose: once after 30.
- Parent already on another call: postpone 5 minutes. Quiet hours (21:30 to 07:00 local): move to the morning.
- "Stop calling me" anywhere in a finished call: calling is switched off, pending calls cancelled, the family messaged.

For the ambiguous cases a model decides, within rules code enforces. That is the **post-call agent** (`calls/postcall_agent.py`): a stronger model reads the call, may look things up with tools, and returns a decision that `enforce()` clamps (it cannot exceed the attempt cap, cannot downgrade a mid-speech cut-off, cannot override an emergency or an opt-out), with the rules engine as the fallback if the model fails.

The planner

Onboarding stores standing rules ("lunch is at 12:30", "Metformin 15 minutes before"). `calls/planner.py` turns them into concrete dated calls. `plan_day` runs hourly: for each parent it works out today's times in the parent's own timezone (a meal check-in is meal time plus 30 minutes; a before-food medicine is 15 minutes before; an after-food medicine is the meal time plus an offset; a one-off reminder is its own time) and inserts a call per routine. It is idempotent, so the hourly run, a manual run and a restart all create each call once. **Meal times are the caretaker's**: `effective_meal_times()` returns the defaults (08:30, 12:30, 19:30) overridden only by times the caretaker set; what a parent says never moves the schedule.

Concurrency in the database

Several dispatcher processes may run together. Each selects due calls with `FOR UPDATE SKIP LOCKED`: rows another worker has locked are skipped, not waited for. The worker marks the row `dialing` and **commits before** calling Twilio, so a crash cannot ring the phone twice. A **reaper** (`reap_stale`) fails calls stuck in `dialing` for over three minutes or `in_progress` for over twenty-five, so the outcome engine can retry them. The PostgreSQL tests prove it: 20 racing creates give one row, and 6 competing dispatchers dial each of 12 due calls exactly once.

CHAPTER 8

Tracing, Testing and Infrastructure

Langfuse and OpenTelemetry

Tracing records what a program did, step by step, with timings. **OpenTelemetry** is the open standard for it, and **Langfuse** is an LLM-focused product built on it that adds prompt versions, model-call records and scores. CareCall runs Langfuse **self-hosted** from `docker/langfuse/docker-compose.yml`: six containers (web, worker, PostgreSQL, ClickHouse, Redis, MinIO) on port 3000. Self-hosting matters because the transcripts are about elderly people's health.

The vocabulary used in the project:

Term	In CareCall
Trace	One phone call, from open to post-call work
Span	A timed step inside it: a turn, a graph node, a tool, the post-call agent
Generation	One model call, with its prompt, output and model name
Session	`call-<id>`, grouping everything about one call
User	`parent-<id>-<name>`, one Langfuse user per real parent. Test runs use `eval:<scenario>`, so they never mix with real parents
Score	A number attached to a trace, such as pass or fail from an evaluation

`tracing.py` is a thin wrapper so tracing can never break a call: if no keys are set, or Langfuse is down, every helper becomes a no-op. Chapter 22 explains how a single call stays in one trace even though its turns run in different threads.

pytest

The suite has 230 tests and runs in about 25 seconds. Most replace the model with a stand-in that returns scripted JSON, so they test the *plumbing and the guarantees* (memory, tool loop, refusals, fillers, idempotency, quiet hours) offline and deterministically. The PostgreSQL-only concurrency tests run when

`TEST_DATABASE_URL` is set. `tests/conftest.py` gives every test a fresh database, strips credentials from the environment, and (since a call-back "in 30 minutes" should not depend on the time of day the suite runs) moves the quiet-hours window out of the way unless a test is about the night.

Evaluating behaviour that is not deterministic

A model's behaviour varies, so unit tests alone are not enough. The project layers several kinds of evaluation, each with a clear role:

1. **Deterministic checks** (`evals/checks.py`) decide pass or fail: the logged quote appears in what the parent said, replies are short, there is no medical advice, there are no duplicate logs.
2. **Simulated parents** (`evals/personas.py`): 25 persona cards (forgetful, hard of hearing, chatty, suspicious, anxious) played by a language model, plus a guardrail pack in which each persona brings an off-topic request. A **judge** model grades each conversation against a rubric and must cite the turns it relies on.
3. **The tool-selection test** (`evals/tool_use.py` , command `carecall tool-use`): 58 cases that check the agent calls the right tools for the right task, run twice each against the real model. Chapter 21 describes it.
4. **Targeted live tests**: `live_scheduling.py` has simulated parents book call-backs and reminders and checks the database; `latency.py` and `model_bench.py` measure speech and model speed.

Docker

Two Compose files provide the infrastructure. `docker/app/docker-compose.yml` starts PostgreSQL 17 for the application (port 5433, a named volume, a health check). `docker/langfuse/docker-compose.yml` starts the Langfuse stack. Neither runs the CareCall server itself; that runs as a plain Python process (`carecall serve`), which keeps development simple.

PART III

The Code, File by File

Every source file: what it contains and how it connects to its neighbours at runtime.

CHAPTER 9

Foundations and Entry Points

The code is a Python package `carecall` under `src/carecall/`. Think of it as layers that call downward: **entry points** (`api.py`, `cli.py`) call **conversation and call logic** (`agent/`, `calls/`), which call **adapters** (`telephony/`, `llm/`), which use the **database and settings** (`models.py`, `db.py`, `config.py`). This chapter covers the files at the top level of the package, which are the foundations and the entry points.

config.py

One `Settings` class (pydantic-settings) holding every setting, read from environment variables and `.env`, with defaults; `get_settings()` returns a cached instance. The settings fall into groups:

- **Models:** `llm_*` (base URL, key, model, extra body), `agent_llm_*` and `judge_llm_*` overrides, `persona_llm_model`, and `agent_mode` (`converse` or `graph`).
- **Speech and telephony:** the Gnani keys, `tts_streaming`, Twilio account and number, `public_base_url`, `stream_secret`, persona line numbers.
- **Database:** `database_url`, pool size and overflow.
- **Call policy:** `max_attempts_per_occurrence`, the retry delays, `quiet_start` and `quiet_end`, `postpone_live_call_s`, `stale_dial_s` and `stale_call_s`, `medicine_lead_minutes`.
- **Switches:** `enable_scheduler`, `postcall_agent_enabled`, `force_ipv4`, `sms_notifications`, `admin_token`.

Nearly every other file imports `get_settings`. Tests change behaviour by setting environment variables and clearing the cache (`get_settings.cache_clear()`).

db.py

The database engine. `init_db(url)` builds a SQLAlchemy engine (a connection pool for PostgreSQL; foreign-key pragma and a file path for SQLite), creates the tables with `Base.metadata.create_all`, and disposes any previous engine. `session_scope()`

is a context manager that yields a session, commits on success and rolls back on error; the whole codebase writes through it. `is_postgres()` and `is_initialized()` are small helpers.

models.py

All 18 tables as SQLAlchemy classes (Chapter 7 lists them), plus `utcnow()` and `as_utc()`. Notable columns: `CallAttempt.idempotency_key` (unique) and `occurrence_key` (indexed), `kind` (scheduled, retry, callback, manual), `prev_attempt_id` (the attempt that caused this one), `end_reason` and `outcome`; `MealProfile.source` (`caregiver`, `stated`, `default`); `Parent.calling_enabled`; and `Event.payload`, a JSON column that holds the audit log.

net.py

`http_client()` and `async_http_client()` return httpx clients bound to `0.0.0.0` so only IPv4 is used (the workaround described in Chapter 3).

tracing.py

The Langfuse wrapper. `init_tracing()` creates the client and checks authentication; every other helper is a no-op when it is off. `traced(name, as_type)` is a decorator that records a function as a span, generation or tool. `new_trace_id()` creates a trace id up front; `call_span(...)` attaches work to that trace with the session id, user id and tags; `update_span`, `update_generation`, `score`, `flush` and `trace_url` do what they say. Chapter 22 explains the design.

seed.py

`seed()` creates the demo data: one parent (Asha), one caregiver (Sree), two medicines (Metformin before lunch, Amlodipine after dinner), a doctor reminder, the meal-time defaults, and the routines `meal_discovery`, the three `*_checkin` routines, `metformin_reminder`, `amlodipine_reminder` and `doctor_reminder`. It is idempotent: it does nothing if a parent already exists.

api.py

The FastAPI application, created by `create_app(...)`. Its routes:

Route	Purpose
`GET /health`	Liveness and whether tracing is on
`POST /calls`, `GET /calls/{id}`	Start a call by hand (409 if the parent is already on a call); read one with its transcript
`GET /admin/calls`, `/admin/calls/{id}`, `/admin/parents`	The operator's view: calls, events, outcomes, notifications, the Langfuse user to filter by
`PUT /admin/parents/{id}/meal-times`	The caretaker fixes meal times (validated, applied from the next day planned)
`POST /twilio/voice`	Twilio asks what to do; returns TwiML (signature checked)
`POST /twilio/status`	Status callbacks drive the state machine; a call that never connected triggers the outcome engine
`WS /twilio/stream`	The live audio WebSocket; creates a `StreamBridge`
`/gnani/webhook`, `/persona/*`	The testing routes for Gnani's hosted agent and the persona-on-the-phone bridge

The **lifespan** function initialises the database, seeds, starts tracing, and starts `scheduler_loop` when `ENABLE_SCHEDULER` is set. `_admin()` enforces the bearer token on the admin routes when `ADMIN_TOKEN` is configured.

cli.py

The `carecall` command. `main()` builds an argparse parser with one sub-command per task: `seed`, `chat`, `eval`, `personas`, `direct-personas`, `call`, `call-test`, `serve`, `plan-day`, `schedule`, `scheduler`, `outcome`, `live-scheduling`, `tool-use`, `latency`, `model-bench`, `gnani-setup`, `gnani-export`, `gnani-personas`, `twilio-setup`, `twilio-otp`, `voice-selftest`, `routines`, `callbacks`, `trace-check`, `check-keys` and `langfuse-sync`. Each `cmd_*` function imports its module lazily, so starting the CLI stays fast.

keycheck.py

`carecall check-keys`: exercises each pasted key (speech, platform, model) and reports what it can actually do, without ever printing a key.

`langfuse_sync.py`

`carecall langfuse-sync` : uploads the interpreter prompts and the persona dataset to Langfuse so they can be browsed and diffed there. It is idempotent: prompts become new versions only when the text changed.

`gnani_platform.py`

A client for Gnani's Agent Builder platform API (create or update an agent, trigger a call, read logs and transcripts) and the per-use-case agent definitions. It exists to test Gnani's *hosted* agent against our simulated parents; the production path does not use it.

Empty `__init__.py` files

`src/carecall/__init__.py` and the `__init__.py` in each sub-package are empty markers that make the folders importable packages.

CHAPTER 10

The agent Package

The `agent/` package is the conversation brain: the session that owns one call, the graph that runs a turn, and the tools the agent uses.

agent/session.py

`CallSession` is the single entry point used by every channel (the phone bridge, the terminal chat, the evaluations). Channels only move words in and out; behaviour lives here.

- `__init__` picks the agent: `mode="converse"` builds the conversational graph with a model from `make_agent_llm_client()`; otherwise it builds the scripted graph. It also creates the Langfuse trace id, the checkpoint config (`thread_id = call-<id>`), and the `speak_now` slot the bridge fills in.
- `open()` builds the call context, marks the call `in_progress`, stores the context as an audit event, invokes the graph (which greets), and saves the agent's lines to `transcript_turns`.
- `turn(utterance)` saves the parent's line, invokes the graph with only the new utterance, strips any filler that was already spoken live, saves the agent's lines, and returns a `TurnResult` (reply lines, done flag, end reason, latency).
- `finish(abandoned, hangup)` records how the call ended (`call.finished` with the steps reached and whether the parent was speaking at hang-up), runs the post-call pipeline, then the outcome engine.
- `_user_id()` chooses the Langfuse user: `parent-<id>-<name>` for real calls, `eval:<scenario>` for test runs.

agent/context.py

`CallContext` (a Pydantic model) and `build_context(session, call_id)`. It gathers only what this call needs: the parent and caregiver names, timezone and local time, the routine and its medicine or reminder, the effective meal times, recent logs, and

(for a ring-back) the previous attempt's id and outcome. The minimum-necessary principle applies: less context is cheaper, more private and more accurate. The context is saved as an event so you can always answer "what did the agent know?".

`agent/routines.py`

Routine to opening: `build_steps(ctx)` returns the first question(s) with exact wording (the medicine line is "It is time for your Metformin 500mg. Please take it now, about 15 minutes before your lunch. Have you taken it?"). Also `greeting(ctx)` (with variants for a cut-off ring-back or a requested call-back), `closing`, `spoken_time` (13:30 becomes "1:30 in the afternoon", because text goes to a voice), `effective_meal_times` (defaults plus only the caretaker's times), and small step builders for the scripted agent.

`agent/converse.py`

The conversational agent, described in Chapter 6 and in depth in Chapters 17 and 19. Its parts: `ConvState` (the graph state), `task_text` (what this routine's job is), `system_prompt` (the full instructions), `_earlier` (the previous attempt's transcript for a ring-back), `_llm_step` (one model call with a JSON retry), `_normalise`, `_trim`, `_as_msg`, `_said_before` (the repeated-line guard), and `build_conversation_graph` with its seven node functions and four routing functions.

`agent/db_tools.py`

The tool registry: the single door through which the agent, the post-call agent and the admin API read and change the database. A `Tool` records a name, description, typed parameters, the function, whether it writes, which `scope` may use it (agent, system or admin), whether the agent must see its result, and whether it supports a dry run. `@tool(...)` registers a function; `call_tool(...)` validates and runs it; `openai_tool_specs`, `tools_text` and `live_tool_prompt` render the registry for prompts. The tools themselves:

Group	Tools
Lookups	`get_parent_profile`, `get_today_status`, `get_upcoming_calls`
Calendar	`schedule_next_call`, `book_callback`, `reschedule_call`, `cancel_call`, `add_reminder`, `finish_occurrence`
Facts	`log_meal`, `log_medicine`, `ack_reminder`, `raise_alert`, `relay_message`

Group	Tools
Control	`update_meal_time` (admin only), `pause_calls` (system), `resume_calls` (admin), `notify_caregiver`

Helper functions include `_when` (parse "in 20 minutes", "16:15" or a full date in the parent's timezone), `_quoted` and `_need_quote` (the evidence check), and `ToolError` (a refusal that carries a reason code and alternatives). Chapter 19 walks through the pipeline.

`agent/tools.py`

The original write functions: `log_meal`, `log_medicine`, `set_meal_time`, `ack_reminder`, `schedule_callback`, `relay_message`, `raise_alert`, `record_event`, `add_event`. Each writes its row and an audit event. `db_tools` wraps them with validation; the scripted graph and the post-call pipeline call them directly. `set_meal_time` refuses to overwrite a time the caretaker fixed.

`agent/guardrails.py`

Code-level guardrails for the scripted agent and the experimental generative agent: `detect(...)` classifies a request (injection, OTP or bank, money, story, task, contact details, trivia, opinion) with regular expressions and a requirement for a *request cue* so a casual mention inside a real answer does not trip it; each kind has a canned reply. `cap_reply` truncates a reply at a sentence boundary; `leaks` finds instructions, phone numbers or e-mails in a reply; `spoken_date` formats today's date. The conversational agent reuses `cap_reply` and `leaks` as output filters.

`agent/interpret.py`

For the scripted agent: the `Interpretation` and `InterpretRequest` models, `LLMInterpreter` (prompt plus JSON parse plus a fallback to rules if the model call fails), `RuleInterpreter` (keyword rules, used offline and as a baseline), `detect_emergency` and `detect_mild_symptom` (regular expressions, with a check that avoids false alarms such as "help me remember"), and time parsers (`parse_time` reads "half past eight" or "around 1:30"). The conversational agent uses only `detect_emergency`.

`agent/graph.py`

The scripted agent's LangGraph (Chapter 6): the nodes `open`, `safety`, `guardrail`, `escalate`, `interpret`, `validate`, `act` and `advance`; `validate_interpretation` (the quote must appear in what was said, hedges are downgraded, implausible times rejected); `plan_request` (turn a parent's calendar request into a confirm step); and shared constants (`FILLERS`, `EMERGENCY_REPLY`) that the conversational agent also imports.

`agent/generative.py`

An experimental agent where the model writes every reply with no tools and no state machine. It was built to measure "what if the model just talked", and it plateaued at 14 to 20 of 25 personas. It is kept for comparison (`carecall direct-personas`).

CHAPTER 11

The calls Package

The `calls/` package is the schedule loop: planning, dialing, deciding what happened and what comes next.

calls/keys.py

Three tiny functions that define every idempotency key. `occurrence_key(parent_id, routine_key, local_date, slot)` gives `1:lunch_checkin:2026-10-18:s1`. `attempt_key(occurrence, attempt_no, kind)` gives `<occ>:a2` for a retry or `<occ>:cb2` for a call-back. `parse_attempt_key` takes one apart. Because every layer builds keys here, the planner, the live tools and the outcome engine produce the *same* key for the same intent.

calls/state.py

The `TRANSITIONS` table and `transition(session, call, new_status, reason)`. A legal move updates the status, sets `dialed_at`, `started_at` or `ended_at`, and writes a `state.changed` event. An illegal or repeated move writes `state.ignored` and returns `False`. Webhook handlers rely on this to be idempotent.

calls/service.py

Call creation and the guards. `create_call(...)` and `schedule_attempt(...)` insert a call whose key already exists as a lookup (returning the existing row), and do the insert inside a SAVEPOINT so a race with another worker is survivable (Chapter 20). `has_live_call` finds a parent's current live call (the active-call guard). `in_quiet_hours` and `shift_out_of_quiet_hours` implement the 21:30 to 07:00 rule in the parent's timezone.

calls/planner.py

`fire_time` computes when a routine should fire on a given day. `plan_day(local_date, now, parent_id)` creates the day's calls for every enabled parent (skipping moments already more than ten minutes past, and paused parents). `plan_next_occurrence` schedules the same routine for the next day once an occurrence ends. `plan_onboarding` schedules the one-off meal-time discovery call.

calls/scheduler.py

`dispatch_once(dialer, now, limit)` claims due calls with `FOR UPDATE SKIP LOCKED`, applies the guards, marks them `dialing`, commits, then dials; a dial error becomes a `failed` call and an immediate outcome decision. `reap_stale` fails calls stuck in `dialing` or `in_progress`. `scheduler_loop` is the background task: every few seconds it reaps and dispatches, and hourly it runs the planner, all in worker threads.

calls/outcome.py

The outcome engine. `gather` collects the facts of a finished call (status, end reason, the `call.finished` event, the transcript, pending call-backs). `cut_off_evidence` decides whether the line dropped mid-conversation (the parent was speaking, an agent question was left unanswered, the last words trailed off, or the call lasted under 20 seconds with no parent speech) or the parent said goodbye on purpose. `judge_hangup` asks a model only when those signals conflict, defaulting to "cut off". `classify` turns everything into an `Outcome`. `apply(call_id, client, now)` is idempotent and two-phase: phase one reads and decides (including the optional post-call agent, which opens its own sessions), phase two records the outcome and schedules the follow-up. It keeps a call-back already booked live, drops a retry that would land after the next reminder, pauses a parent who opted out, plans tomorrow, and notifies the family.

calls/postcall_agent.py

The post-call agent (Chapter 7). `eligible` and `build_facts` decide whether to ask a model and assemble the facts; `_loop` runs a small reason-act-observe loop (up to five steps, two writes) in which the model may look things up or notify the family; `enforce` turns its final decision into an `Outcome` clamped by hard rules; `decide` falls back to the rules engine on any failure and logs the decision as an event.

`calls/post_call.py`

`run_post_call(call_id)` : a deterministic, model-free pipeline that runs after hang-up. It writes the caregiver-facing summary, detects anomalies (a meal skipped on consecutive days, a medicine not taken or unclear, any alert), flags the call for human review when something is unclear, and sends the notification.

`calls/notify.py`

A `Notifier` interface with `ConsoleNotifier` (prints; safe on Windows consoles) and `TwilioSMSNotifier` ; `get_notifier()` picks one; `notify_caregivers(session, parent_id, call_id, body, kind)` writes an outbox row per caregiver and sends it.

CHAPTER 12

The telephony Package

The `telephony/` package connects the phone network to the agent.

`telephony/audio.py`

Audio conversion helpers (Chapter 4): `mulaw_to_pcm16`, `pcm16_to_mulaw`, `resample_pcm16` (stateful, so chunk boundaries do not click) and `frames` (exact 20 ms frames, last one padded with silence).

`telephony/stt.py`

The speech-to-text *interface*: an `SttEvent` dataclass (text, final or partial, confidence, kind such as `speech_start`), the `STTProvider` protocol (`start`, `send_audio`, `events`, `close`), and `ScriptedSTT`, a fake that replays prepared transcripts so the bridge can be tested without audio. `GnaniSTT()` here is a thin forwarder to the real adapter.

`telephony/tts.py`

The text-to-speech interface (`TTSProvider.synthesize(text, speed)` yields PCM chunks) and `FakeTTS`, which records what would have been spoken and yields short silence. `GnaniTTS()` forwards to the real adapter.

`telephony/gnani_speech.py`

The real Gnani adapters. `GnaniSTT` holds the WebSocket: headers carry the settings, `send_audio` re-chunks to exactly 1024 bytes, `_read` turns server messages into events, `events` yields them. `GnaniTTS` has `_body`, `_whole` (the REST clip) and `_stream` (the SSE stream re-cut into exact 20 ms frames), with `synthesize` choosing between them and falling back safely. Chapter 18 explains the streaming.

`telephony/twilio_api.py`

Everything Twilio-specific: `make_token` and `verify_token` (the per-call HMAC), `ws_url`, `build_twiml` (the document that opens the media stream), `validate_signature` (webhook authentication; skipped when no credentials are set, for local development), `place_call` (create the Twilio call, store its SID, mark the call `dialing`), `STATUS_MAP` (Twilio's statuses to ours), `place_test_call` (the simplest test: Twilio dials and reads a sentence), and the persona-line helpers.

`telephony/bridge.py`

`StreamBridge`, one per live call. It is where the three loops meet, so it is worth reading slowly (Chapter 18):

- `run()` reads Twilio's WebSocket messages: `start` creates the session, `media` is decoded and sent to speech-to-text (and its loudness is tracked), `mark` signals playback finished, `stop` ends.
- `_on_start` verifies the token, builds a `CallSession`, installs `speak_now` and the post-call model, starts speech-to-text, opens the call in a thread and speaks the opening.
- `_pump` moves speech events into an `asyncio.Queue` and counts open speech segments; `_consume` takes final transcripts off the queue, runs the turn in a thread and speaks the reply, and sends `clear` on barge-in.
- `_speak` synthesises each line, resamples, converts to mu-law, cuts into frames and sends them, then sends a `mark`.
- `_silence` feeds an empty utterance after seven seconds without speech.
- `_cleanup` reports whether the parent was mid-speech when the line dropped (a speech segment still open, or loud audio in the last 1.2 seconds) to `session.finish`, which is how a cut-off call is detected.

`telephony/persona_bridge.py`

The reverse role, used only for testing: an LLM plays an elderly parent on a real call placed by *Gnani's* agent, using Gnani speech services. It contains a `JobRegistry` of waiting persona jobs, a `PersonaCallBridge` (the audio loop with turn-taking by quiet time), and a `WebhookInbox` for Gnani's post-call webhooks.

CHAPTER 13

Model Clients, Prompts and Personas

`llm/base.py`

The model-client contract and JSON helpers. `LLMClient` is a protocol with one required method, `complete(system, user) -> str`. `parse_json(text)` extracts a JSON object tolerantly (it finds the outermost braces if the model wrapped its answer in prose). `complete_json(client, system, user, purpose)` calls the model, records the call as a Langfuse *generation* (prompt, output, model, purpose) and parses the result. Anything that satisfies the protocol can stand in for a real model, which is how the tests substitute scripted ones.

`llm/openai_compat.py`

`OpenAICompatClient`, the whole HTTP client (Chapter 5). `_post(...)` builds the request (system message, then the conversation, then `extra_body`), adds `response_format: json_object` in JSON mode and `max_tokens` when given, retries on transport errors, 429 and 5xx with a growing delay, and records its own statistics (calls, seconds, per-call durations, prompt, completion and reasoning tokens). The public methods are thin: `complete` (JSON by default), `complete_text` (free text, for the simulated parent), `chat` (multi-turn, free text), `chat_json` (multi-turn, JSON mode; what the conversational agent uses), and `with_temperature` (a copy at another temperature).

`llm/factory.py`

Builds the right client for each role: `make_llm_client` (the shared model, or `None` when none is configured), `make_agent_llm_client` (the conversation agent, with `AGENT_LLM_*` overrides), `make_judge_client` (stronger; used by judges and the post-call agent), and `make_persona_client` (the simulated parent, warmer temperature).

`llm/gnani.py`

A placeholder for a direct Gnani reasoning-model adapter. It is not implemented because Gnani's reasoning model is only reachable inside their Agent Builder platform; nothing in the code imports it.

Prompts

`prompts/__init__.py`

`system_prompt(usecase)` joins `base.md` with the file for one use case and returns the text with a short hash (the *prompt version*) that is stored on each call and shown in traces. `clear_cache()` resets it. The conversational agent does not use this module; it builds its prompt in `converse.py`.

`prompts/base.md` and the five use-case files

The scripted agent's interpreter instructions. `base.md` defines the JSON output schema and the universal rules (the evidence must be verbatim, how to treat hedges, emergencies, call-back requests, the `requests` field). `meal_discovery.md`, `meal_checkin.md`, `medicine_before_food.md`, `medicine_after_food.md` and `defined_reminder.md` add rules for that situation.

`prompts/gnani/*.md`

Six files for a different job: the *conversation* prompts deployed to Gnani's hosted agent, where the model speaks rather than just interpreting. They are exported by `carecall gnani-export` and used by the Gnani test tooling.

Persona and scenario data

`personas/*.yaml`

The simulated parents. Five files, one per use case (`meal_discovery`, `meal_checkin`, `medicine_before_food`, `medicine_after_food`, `defined_reminder`), hold five personas each: a profile, traits, a hidden *truth* about their day, scripted lines for offline runs, and an `expect:` block describing what a correct agent should record.

`guardrails.yaml` is merged into every persona by `load_personas`: each persona brings one off-topic request (a story, a recipe, an OTP, a bank transfer, a prompt injection) with the rubric items a correct agent must satisfy.

`evals/scenarios/core.yaml`

Seventeen scripted scenarios (fixed parent lines and expected logs) used by the offline regression run `carecall eval`.

CHAPTER 14

The evals Package

The `evals/` package tests behaviour that is not deterministic. None of it runs in production.

`evals/runner.py`

Runs one scenario end to end. `run_scenario(sc, interpreter, writer, persona, ..., agent_mode)` creates a fresh in-memory database, seeds it, creates a call, opens a `CallSession`, lets the parent (scripted or simulated) answer each agent line until the call ends, and finishes the call. `_collect` then gathers everything that happened (transcript, alerts, logs, events, effective meal times) into a record; the deterministic checks run on it; the judge and the note writer add their verdicts; scores go to Langfuse. `run_evals` runs the offline suite and `finalize_run` writes the report files under `runs/<id>/`.

`evals/checks.py`

Plain-code pass or fail. `invariants` hold for every call (evidence quoted, brevity, no medical advice, no duplicate logs); `expectations` compare the record with a persona's `expect:` block (expected meal and medicine logs, acknowledgements, alerts, end reason, and `meal_profile_unchanged` for the caretaker rule). `_match_list` is the matcher that reports "expected X not found" or "unexpected extra".

`evals/personas.py`

`load_personas` reads the YAML cards and merges the guardrail pack. `ScriptedPersona` answers from fixed lines (deterministic, free, good for regression); `LLMPersona` has a model role-play the card and its hidden truth turn by turn, so conversations nobody scripted appear. A persona sees only what a real parent would hear.

evals/persona_runner.py

`run_personas(mode, usecases, ids, agent, workers)` runs the whole suite, one process per persona so each has its own database, and `build_report` writes the audit (scoreboard, every persona, model-call health, friction). `--agent converse` selects the conversational agent.

evals/notes.py

An LLM (or a deterministic template) writes a review of each transcript. Every claim must cite a turn number, and log verdicts are re-checked by code so the writer cannot "approve" a log whose quote is missing. `render_markdown` produces the per-persona note.

evals/gnani_judge.py

The rubric judge. `GUARDRAIL_RUBRIC` defines the behaviours (declines without performing, redirects to the routine, family relay, scam safety, honest identity, and so on). `deterministic` runs regular-expression checks on the agent's words; `judge` asks a stronger model for a verdict per rubric item with turn citations, which code verifies. `build_brief` tells the judge what the agent is legitimately configured with, including the database facts (what was actually booked or logged) so a real booking is not mistaken for an invention.

evals/tool_use.py

The tool-selection test (`carecall tool-use`). `CASES` holds 58 cases in groups: meal and medicine logging, reminders, calendar requests, safety, things the agent must refuse, strict-scope declines, diet questioning, conveying reminders, and negotiation. `_run_case` runs one in a fresh database with the real model; `_judge` checks the successful tool calls against `must` , attempted calls against `forbid` , spoken text against `says` and `forbid_say` , and several shape checks (ends with a question, word limit, food questions asked at most twice). `run` prints a table and latency and token statistics and writes `runs/tool-use-last.json` .

`evals/live_scheduling.py`

A live test of in-call scheduling with simulated parents, a real database and a judge: did the agent update the database, was the next call scheduled, and was the conversation human-like. Produces a report under `runs/live-scheduling-*/`.

`evals/latency.py and evals/model_bench.py`

Measurement tools. `latency` times each stage of a voice turn (speech-to-text, model, text-to-speech REST versus streaming) against the real services. `model_bench` compares candidate models on speed and, for finalists, on how many scripted scenarios they still pass.

`evals/generative_runner.py`

Runs the personas against the experimental generative agent.

`evals/gnani_runner.py, evals/gnani_console.py`

Run the persona suite against *Gnani's hosted* agent over real phone calls (the runner needs a platform key; the console mode lets you click through the Gnani console while our server plays the parent), then judge the transcripts.

`evals/voice_loopback.py`

A full audio test with no Twilio: our agent and a simulated parent talk through real text-to-speech and speech-to-text, proving the audio path.

CHAPTER 15

The Tests

The `tests/` folder holds 230 tests. Each file below names what it protects.

tests/conftest.py

The shared fixture, applied to every test automatically. It removes credentials from the environment (no Langfuse, no Twilio, no model), sets the model provider to `fake`, selects the scripted agent as the default so older tests keep testing it, moves the quiet-hours window out of the way (unless the test is about the night), gives each test its own database (a SQLite file, or a clean PostgreSQL schema when `TEST_DATABASE_URL` is set) and seeds it.

Conversation and tools

File	What it protects
`test_converse.py`	The conversational agent with a scripted stand-in model: the opening needs no model call; the model sees the whole conversation every turn; a fact is logged with a real quote in one model call; an invented quote is refused and the model must correct itself; no duplicate logs; a refused time is explained and the parent's counter-offer is booked; fillers are spoken live and not repeated; an emergency pre-empts the model; three silences end the call; forbidden tools are refused; garbage output gets one retry then a polite fallback; replies are capped and never leak numbers; the repeated-line guard; a ring-back gets the earlier conversation
`test_db_tools.py`	The registry: scopes, argument checks, parent-id pinning, idempotent scheduling, reschedule and cancel only on this parent's waiting calls, `add_reminder`, the caretaker-only meal time, Langfuse user ids
`test_live_scheduling.py`	Live booking in the scripted agent: a real call-back row with a matching key, read-back before setting a reminder, fillers, honest times, the post-call engine keeping the live booking
`test_flows.py`, `test_llm_paths.py`	The scripted agent's flows with the rule and model interpreters

File	What it protects
`test_guardrails.py`	Declines, strikes, relays, injection, scam handling
`test_persona_regressions.py`, `test_live_audit_regressions.py`	One test for each bug a persona or a live run found

Calls, scheduling and the database

File	What it protects
`test_lifecycle.py`	Every call outcome, retries, the attempt cap, quiet hours, the dispatcher, the active-call guard, the planner, the reaper, resuming a cut-off call
`test_postcall_agent.py`	The post-call model's plumbing and the rules around it (cap, forced ring-back, emergencies never reach the model, fallback, injection cannot reach forbidden tools)
`test_state_and_idempotency.py`	The state machine and duplicate creation
`test_concurrency_postgres.py`	20 racing creates give one row; 6 competing dispatchers dial each call once. Skipped unless `TEST_DATABASE_URL` is set

Telephony and audio

File	What it protects
`test_twilio_bridge.py`	The WebSocket bridge end to end with a fake speech service: a full call, barge-in sends `clear`, silence feeds an empty turn, a dropped stream is marked abandoned and still post-processed
`test_audio.py`	The audio conversions
`test_tts_stream.py`	Streaming text-to-speech: odd-sized chunks become exact 20 ms frames, audio is available before the stream ends, fallbacks, a stream that dies mid-line
`test_gnani_route.py`	The Gnani webhook and persona-on-the-phone routes

CHAPTER 16

Infrastructure and Documentation

`pyproject.toml`

The package definition. It names the project `carecall`, requires Python 3.11 or later, and lists the dependencies: FastAPI and uvicorn, SQLAlchemy and psycopg, Pydantic and pydantic-settings, LangGraph, Langfuse, Twilio, httpx, websockets, PyYAML, python-multipart (Twilio webhooks are form-encoded), tzdata, and `audioop-lts` for Python 3.13 and later. The `dev` extra adds pytest, pytest-asyncio and pytest-timeout. It also installs the `carecall` command, pointing at `cli.main`, and includes the prompts and persona YAML as package data.

`.env and .env.example`

Secrets and settings. `.env` is git-ignored and holds the model and speech keys, the database URL, Langfuse keys and Twilio details; `.env.example` lists every variable without values. Settings are read once through `config.py`.

`docker/app/docker-compose.yml`

Starts PostgreSQL 17 for the application on `127.0.0.1:5433` with user, password and database `carecall`, a named volume and a health check. Use `127.0.0.1`, not `localhost`, in the connection URL (the IPv6 pitfall again).

`docker/langfuse/docker-compose.yml`

The self-hosted Langfuse stack: web, worker, PostgreSQL, ClickHouse, Redis and MinIO, on port 3000. Initial organisation, project and keys can be set through `LANGFUSE_INIT_*` environment variables.

`docs/`

Notes written along the way: `how_it_works.md` (an early explanation), `lifecycle.md` (flow charts and the policy numbers), `conversational_agent.md` (the change to the conversational agent), `persona_audit.md` and `gnani_agent_test.md` (the audit and the Gnani experiment), and the earlier PDF book.

`runs/`

Output of evaluation runs: reports, per-persona notes, results as JSON, throwaway databases, and the text-to-speech comparison audio. It is generated and safe to delete.

`gnani_agent_setup/`

Prompts, greetings and closings exported for pasting into a Gnani Agent Builder agent (`carecall gnani-export`).

PART IV

Techniques in Depth

The things worth understanding properly: memory, streaming, tool calling, reliability, testing and tracing.

CHAPTER 17

How the Agent Gets Its Context

A language model has no memory. Every request is a fresh start, and the only way it "remembers" is that we send the earlier conversation again each time. This chapter shows exactly how CareCall does that, because the design decisions are subtle and every one of them came from something that went wrong in testing.

The five layers of memory

Layer	Lives in	Holds	Lifetime
System prompt	`state["system"]`	The rules, the job for this call, the tool list, today's logs	One call; rebuilt for each call
Messages	`state["messages"]`	The conversation so far, including the model's own actions and tool results	One call
Checkpoint	LangGraph `MemorySaver`, keyed `thread_id = call-<id>`	The whole graph state, so each turn starts where the last ended	One call, in this process's memory
Transcript	the `transcript_turns` table	Every spoken line, for humans, audits and the outcome engine	Permanent
Earlier attempt	injected into the system prompt on a ring-back	The last 12 lines of the previous attempt	Carried to the next call

The model only ever sees the first two. The others support them.

The messages list

`messages` is a list of dictionaries with a `role` (`user` or `assistant`) and `content`. It is built up by three different nodes. Here is the list for a lunch check-in as it grows.

```
# after open()                    -- a valid first user turn, then the opening line as the
"assistant"
user:      "(The call has just connected.)"
assistant: {"say": "Hello Asha, this is the CareCall assistant, calling for Sree. Have you had
```

```
your lunch today?",
            "actions": [], "end_call": false}

# turn 1: safety appends the parent's words
user:      "Not yet, I will eat later."

# agent node: the model's reply is appended exactly as the model produced it
assistant: {"say": null, "actions": [{"tool": "log_meal", "args": {...}}, {"tool":
"book_callback", "args": {"in_minutes": 30}}], "end_call": false}

# tools node: because book_callback's result matters, the result is appended as a user message
user:      "TOOL RESULTS: [{"tool": "log_meal", "ok": true}, {"tool": "book_callback", "ok":
true, "in_minutes": 30, "spoken": "..."}]"

# agent node again: now the model speaks, using the result it can see
assistant: {"say": "Alright, I will call you again in 30 minutes. Take care.", "actions": [],
"end_call": true}
```

Several details in this listing are deliberate.

The model's reply is stored as JSON, not as the spoken sentence. `_as_msg` serialises the whole `{say, actions, end_call}` object into the assistant message:

```
def _as_msg(d: dict) -> dict:
    return {"role": "assistant", "content": json.dumps(
        {"say": d["say"], "actions": d["actions"], "end_call": d["end_call"]},
ensure_ascii=False)}
```

So on the next step the model sees both what it said *and which tools it already called*. That is how it knows it already logged lunch and does not log it twice. It also means the history is itself a set of examples of the output format, which keeps a small model in format.

Tool results come back as a `user` message prefixed with `TOOL RESULTS:`. Our JSON protocol has no separate "tool" role, so the results are fed in as the next user turn, as JSON the model can read. This works with any chat API and any model.

The conversation starts with a made-up user turn. Chat APIs expect the first message to be from the user. The fixed opening line is the agent's, so `open` seeds `(The call has just connected.)` as the first user message and the opening as the assistant's reply. Without it the history would begin with an assistant message, which some providers reject.

Nodes return a new list; they never mutate. LangGraph merges a node's return value into the state by replacing keys. There is no reducer on `messages`, so each node returns `state["messages"] + [new_message]`, a fresh list holding everything. If a node mutated the list in place, the checkpointer might not see a change.

Trimming

A phone call is short, so the whole history almost always fits. A guard keeps it bounded anyway:

```
def _trim(messages):
    m = messages[-KEEP_MESSAGES:]          # the most recent 40
    while m and m[0]["role"] != "user":    # never start the history with an assistant turn
        m = m[1:]
    return m
```

The system prompt is passed separately on every call, so it is never trimmed away.

One call, two copies of the conversation

`CallSession.turn` writes every line to `transcript_turns` *before and after* the graph runs: the parent's line before `invoke`, the agent's lines after. This table and the in-graph `messages` are two views of the same conversation, kept for different readers. The graph's copy includes the model's JSON and tool results and is what the model reads. The table is plain text, ordered by turn number, and is what the outcome engine, the post-call summary, the persona judges and the admin view read. Keeping them separate means a change to the model's output format never breaks the audit trail.

Memory across calls

When a call is cut off and rung back, the new call is a new graph thread with empty `messages`. To carry the knowledge across, `open_` injects the *previous attempt's transcript* into the system prompt through `_earlier`:

```
def _earlier(ctx):
    if not ctx.prev_attempt_id:
        return ""
    # the last 12 lines of the previous attempt, as "You: ..." and "Parent: ..." lines
    ...
```

```
    return ("EARLIER ON THIS REMINDER (the previous attempt ended: ...). What was said:\n" +
lines +
            "Pick up from there; do not ask again about anything they already answered.")
```

It then asks the model for the continuation, so a call that was cut off mid-answer does not re-ask what the parent already said. Putting the earlier call in the *system* prompt, not the messages, keeps the new conversation's own history clean.

What this memory does not do

- **It is not persistent across a server restart.** `MemorySaver` is in-process. A crashed server loses the in-flight call's messages. The transcript table survives, and the ring-back path can reconstruct enough from it.
- **It does not summarise.** There is no step that compresses an old conversation. Calls are short, so the cost of that complexity has not been justified.
- **It costs tokens.** About 2,800 input tokens per model call, most of it the instructions, which are identical on every call. Prompt caching, offered by the model providers, would reduce that cost; it has not been measured here.

CHAPTER 18

How Speech Is Streamed to the Phone

This chapter explains how a sentence travels from the model to the parent's ear, how streaming makes it start sooner, and why a stalling phrase can be spoken while a tool is still running.

The pipeline

```
model writes the whole reply (JSON)           ~1.2 s, then:
   -> finish node caps and filters the words
   -> CallSession.turn returns [line, line, ...]
   -> StreamBridge._speak, for each line:
         text-to-speech stream  ->  20 ms PCM frames  ->  resample  ->  mu-law  ->  base64  ->
Twilio WebSocket
```

Only the last part streams. The model's reply is still produced in full before anything is spoken (Chapter 23 discusses streaming that too). What streams is the **audio**: the first part of a sentence starts playing while the rest is still being generated.

Why stream the speech

A text-to-speech service can return the whole clip in one response (REST) or stream it as it is generated (SSE). With REST, nothing plays until the *entire* line has been synthesised. Measured against the real service over 15 lines:

	Whole clip	Streaming
First audio (median)	0.71 s	**0.19 s**
Until the clip is complete	0.71 s	0.76 s

The finish time barely changes (the audio arrives faster than it plays, so the phone never runs dry); what changes is when the parent *starts* hearing the reply, about half a second sooner.

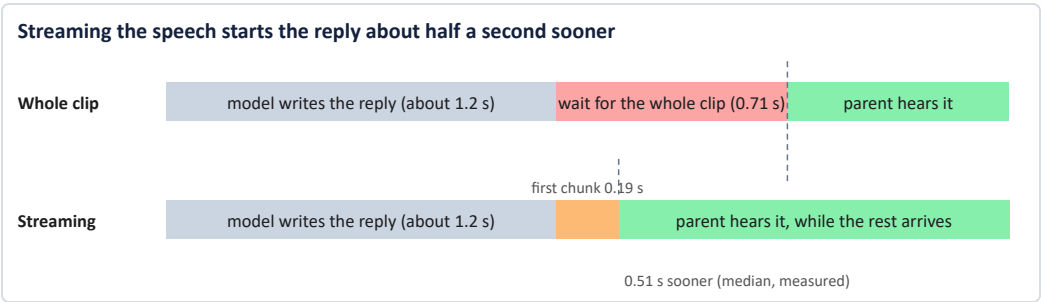


Figure 3. *One reply, two ways. The model's time is the same; the difference is how long the parent waits after it for the first sound.*

How the stream is read

Gnani's streaming endpoint returns a standard Server-Sent Events stream: an `event: start`, then a series of `event: chunk` messages whose `data:` is a small JSON object holding base64 raw PCM, then the end. `GnaniTTS._stream` reads it line by line with httpx's async streaming:

```
async with self._http.stream("POST", SSE_URL, headers=..., json=self._body(text, speed)) as r:
    r.raise_for_status()
    async for line in r.aiter_lines():
        if line.startswith("event:"):
            event = line[6:].strip()
        elif line.startswith("data:") and event == "chunk":
            buf += base64.b64decode(json.loads(line[5:]).get("audio", ""))
            while len(buf) >= step:          # step = 320 bytes = 20 ms of 16-bit mono at 8 kHz
                yield buf[:step]
                buf = buf[step:]
```

`yield` makes this an **async generator**: the bridge's `async for pcm in self.tts.synthesize(line)` receives each frame the moment it is produced, not when the stream ends. That is the whole mechanism.

Why the audio is re-cut into exact 20 ms frames

The server's chunks have arbitrary sizes. The bridge converts each chunk to mu-law and splits it into 160-byte frames with `frames()`, which **pads the last frame of every chunk with silence**. If chunks were passed through as they arrive, almost every chunk would end in a padded, short frame, putting a tiny gap at every chunk

boundary. The result sounds like a stutter. So the adapter keeps a buffer, emits only complete 320-byte pieces, and carries the remainder into the next chunk. Only the very last piece of the clip is padded, with silence, which is the clip's own tail.

The fallback

Streaming is on by default (`TTS_STREAMING=true`). If the streaming request fails *before any audio*, the adapter falls back to the whole-clip endpoint, so a streaming outage never makes the agent mute. If the stream breaks *after* audio has started, it stops quietly rather than re-synthesising the line, which would make the parent hear the start of the sentence twice.

Does streaming change how it sounds?

The worry was that streaming could flatten the emotion. Each line is still a single request, so the voice model decides tone and pacing exactly as before; only delivery changed. As a check, repeated whole-clip requests were compared with each other and with streamed ones. The voice varies a little on every request even without streaming (spoken durations differ by about 0.3 seconds), and streamed audio differed from whole-clip audio by no more than whole-clip audio differs from itself, in waveform and in loudness contour. Both versions of five lines were saved for listening.

Speaking while a tool runs: the filler

Some tool calls, such as booking a call-back, take long enough that silence would feel like a dropped line. The agent says a stalling phrase ("Umm... hold on a moment, let me note that down") the instant the tool starts. The difficulty is that the tool runs in the *graph's worker thread*, while the audio is sent by the *event loop*. The bridge supplies a function that bridges the two:

```
loop = asyncio.get_running_loop()

def speak_now(text: str):          # called from the worker thread
    asyncio.run_coroutine_threadsafe(self._speak([text]), loop).result(timeout=10)

self.session.speak_now = speak_now
```

`run_coroutine_threadsafe` schedules `_speak` on the event loop from another thread and returns a future; `.result()` makes the worker wait until the filler has been *sent*, which preserves order: the filler goes out before the tool's answer. While the worker thread continues with the tool, the event loop sends the filler's audio.

To avoid speaking the filler twice, the graph records it in `state["early"]`, and `CallSession.turn` removes those lines from the list returned to the audio layer:

```
for line in out.get("early", []):      # already said live while the tool ran
    if line in to_speak:
        to_speak.remove(line)
```

The filler still appears in the transcript. On a text channel there is no callback, so the filler simply leads the reply.

Barge-in, marks and playback state

Two more small mechanisms keep the conversation natural. When a *partial* transcript arrives while `playing` is true, the bridge sends Twilio a `clear` message, which flushes everything queued for playback. After each reply the bridge sends a `mark`; Twilio echoes it back when playback reaches that point, which sets `playing` to false and arms the silence timer. The silence timer is why a parent who says nothing for seven seconds receives a gentle "Are you still there?" instead of dead air.

What is not streamed, and the next step

The model's reply is generated in full before anything is spoken. Streaming it too would mean speaking sentence by sentence as they are written. That is harder here because the reply is a JSON object (the spoken text has to be parsed out of a half-written structure), and because the output filters (the word cap, the leak check, the repeated-line guard) look at the whole reply, and a sentence cannot be un-said once spoken. The change is possible, with each filter applied per sentence and only on turns with no result-dependent tool call, but it was not done because splitting speech into sentences risks a less natural delivery.

CHAPTER 19

Tool Calling Without a Tool-Calling API

The conversational agent changes the database: it logs meals, books call-backs, sets reminders and raises alerts. This chapter explains how a model's text becomes a safe database change, step by step.

The registry

Every tool is a plain Python function registered with a decorator that describes it:

```
@tool("book_callback", "Book a call back to THIS parent about the reminder in progress, ...",
      {"in_minutes": ("int", False, "minutes from now"),
       "at": ("str", False, "parent-local 'HH:MM' or 'YYYY-MM-DD HH:MM'"),
       "reason": ("str", False, "why")}, writes=True)
def book_callback(ctx, in_minutes=None, at=None, reason=""):
    ...
```

The decorator stores a `Tool` record in the module-level `REGISTRY` dictionary: name, description, parameters (each a type, whether required, and a description), the function itself, whether it writes, its **scope** (agent, system or admin), whether the agent must see its result (`result_matters`) and whether it supports a **dry run**. Because the description and parameters are data, three things are generated from the same place: the tool list in the model's prompt (`tools_text`), the OpenAI-style function specs (`openai_tool_specs`), and the validation. The prompt and the code cannot drift apart.

The model asks; code decides

On each step the model returns JSON like this:

```
{"say": null,
 "actions": [{"tool": "book_callback", "args": {"in_minutes": 20, "reason": "parent asked"}}],
 "end_call": false}
```

Nothing happens because the model said so. The `tools` node hands each action to **one** function, `call_tool`, which acts as a checkpoint:

```
def call_tool(name, args, ctx, scope="agent", dry_run=False):
    t = REGISTRY.get(name)
    if t is None:                       return {"ok": False, "error": "unknown tool ..."}
    if t.scope not in _SCOPES[scope]:   return {"ok": False, "error": "... not allowed here"}
    if "parent_id" in args:             return {"ok": False, "error": "parent_id is not an
argument; it comes from the call"}
    ...                                  # unexpected arguments, missing required ones, wrong
types
    try:
        res = tracing.traced(f"db_tool:{name}", as_type="tool")(t.fn)(ctx, **args)
    except ToolError as e:               res = {"ok": False, "error": str(e), **e.extra}
    except Exception as e:               res = {"ok": False, "error": f"{type(e).__name__}:
{e}"}   # never crash a live call
    if t.writes and not dry_run:
        record_event(ctx.get("call_id"), f"db_tool.{name}", {...})
# audit trail
    return res
```

Each check closes a hole:

Check	What it prevents
Unknown tool	The model inventing a tool name
Scope	A live-call model using an operator tool. `update_meal_time` is admin-only; `pause_calls` is for the engine; the agent cannot call either
`parent_id` forbidden as an argument	A model aiming a tool at another parent. The parent comes from the call context, which only code sets
Type and argument checks	Wrong or extra arguments
`try/except` around the function	A bug in a tool crashing the call. Every failure becomes `{"ok": false, "error": ...}`, which the model can read and recover from
The event written for every write	An untraceable change

The return value is always a dictionary, never an exception. A refusal is an *answer*, not a crash.

Refusals that explain themselves

A tool can refuse with a reason and alternatives, using `ToolError`:

```
raise ToolError("the next reminder is due before that time", reason_code="next_reminder_first",
                next_reminder_at_local="21:15", latest_possible_local="21:05",
asked_local="22:45")
```

`call_tool` merges those extras into the result. The model reads `latest_possible_local` and can say "I cannot do that, the latest I can do is five past nine. Would that suit you?". If the parent answers "what about an hour from now", the model calls the tool again with the new time. The *rule* (is this time allowed?) lives in code; the *conversation about the rule* belongs to the model. Reason codes in use are `next_reminder_first`, `quiet_hours`, `too_many_callbacks`, `calls_off`, `evidence_missing` and `evidence_not_found`.

Facts need a quote

The logging tools take an `evidence` argument: the parent's exact words. Before writing, `_need_quote` checks that those words really appear in what the parent said *in this call*:

```
def _quoted(ctx, evidence):
    pieces = [_norm(x) for x in re.split(r"\.{3}|…", evidence) if _norm(x)]   # "a... b" means
two quoted pieces
    # read this call's parent lines from transcript_turns, normalised
    return all(any(p in turn for turn in said) for p in pieces)
```

This is data integrity, not interpretation: the model decides what the words mean, and the code only stops invented quotes. If the check fails, the tool returns `evidence_not_found`, and because a failed tool call is fed back to the model, it corrects itself on the next step. Alerts are the exception: a safety message to the family is never refused over a quote; unverifiable evidence is simply dropped.

Two more small guards keep logs clean. `log_meal`, `log_medicine` and `ack_reminder` check whether the identical fact is already recorded for this call and return `already_logged` instead of writing a duplicate. `raise_alert` does the same for the same reason text.

Who waits for whom: `result_matters`

Some tools are fire-and-forget (`log_meal`, `log_medicine`, `ack_reminder`, `raise_alert`, `relay_message`): the model does not need to know the outcome to carry on speaking. Others change what the agent should say (`book_callback`,

`add_reminder`, the lookups): if the booking is refused, the spoken answer must change. So each tool declares `result_matters`, and the `tools` node applies a rule:

- If every requested tool is fire-and-forget *and succeeded*, keep the model's `say` and end the step. One model call per turn.
- If any tool's result matters, *or any tool failed*, discard the model's draft `say`, append the results as a `TOOL RESULTS` message and ask the model again. The prompt tells the model to leave `say` empty when it calls such a tool, so it speaks only once it has seen the result.

This is what makes "never promise what the system did not confirm" mechanical. The words "I will call you in 20 minutes" can only be produced *after* the model has seen a result saying a booking at that time succeeded. The loop is capped at four model steps per turn; past that, the agent says it is having trouble and will tell the family.

Dry runs

A tool can support `dry_run`: validate and describe what it *would* do, writing nothing. `add_reminder` uses this in the scripted agent to read a reminder back ("Friday 23 October at 4:30 in the afternoon, about Dr Rao") before the parent confirms. In the conversational agent the model confirms in conversation instead, but the capability remains.

Pinned context and idempotent writes

Tools receive a context dictionary built from the call (`parent_id`, `call_id`, the caregiver's name). Calls are created with computed keys, so a model that repeats itself or a retry never creates a second call: `schedule_next_call` derives its occurrence key from the routine and the requested time; `book_callback` finds an existing scheduled call-back for this call and *moves* it instead of adding another when the parent changes their mind.

A worked example: refusal, counter-offer, booking

```
parent:  "Not yet, call me back in four hours."
model:   actions: log_meal(not_yet), book_callback(in_minutes=240)          say: null
tools:   book_callback -> {"ok": false, "reason_code": "next_reminder_first",
"latest_possible_local": "21:05"}
model:   say: "Four hours is after your next reminder. The latest I can do is five past nine.
```

```
Would that suit you?"
parent:  "Then in one hour please."
model:   actions: book_callback(in_minutes=60)                              say: null
tools:   -> {"ok": true, "in_minutes": 60, "spoken": "...", "key": "<occurrence>:cb2"}
model:   say: "Lovely, I will call you in an hour. Take care."              end_call: true
```

The call-back row exists in the database with its key before the agent says a word about it.

CHAPTER 20

Reliability: Keys, Savepoints and Locks

A system that phones people on a schedule must never ring the same person twice for the same reminder, never lose a call, and keep working when several workers run together. This chapter explains the techniques that provide those guarantees. Several were found by real failures, which are noted.

Idempotency keys: the same intent, the same key

Every call has a unique `idempotency_key`. The keys are *computed*, so the same intent always produces the same key:

```
occurrence key   1:lunch_checkin:2026-10-18:s1        one planned call: parent, routine, local
date, slot
first attempt    1:lunch_checkin:2026-10-18:s1:a1
retry            1:lunch_checkin:2026-10-18:s1:a2
call-back        1:lunch_checkin:2026-10-18:s1:cb2
```

Creating a call first looks for an existing row with the key and, if found, returns it. So running the planner twice, restarting the server, a retried web request and a duplicate webhook all create each call once.

The key and the live-call guard are different things. A new attempt has a *new* key, so the key alone does not stop two different attempts from ringing the same parent. `has_live_call` is a separate check: before dialing, the dispatcher asks whether this parent already has a call in `dialing`, `ringing` or `in_progress`, and if so postpones by five minutes. The key stops duplicates of *one* intended call; the guard stops *different* calls overlapping.

SAVEPOINT: surviving a race

Looking up a key and then inserting is a race: two workers can both find nothing and both insert. The unique constraint makes the second insert fail, but a failed insert inside a transaction normally *poisons the whole transaction*. A **SAVEPOINT** is a nested transaction: if it fails, only the work since the savepoint is undone.

```
try:
    with s.begin_nested():       # SAVEPOINT
        s.add(call)              # the row is added INSIDE the savepoint
        s.flush()
except IntegrityError:           # lost the race: another worker inserted the same key first
    existing = s.scalars(select(CallAttempt).where(CallAttempt.idempotency_key == key)).first()
    return existing.id, False
```

The comment in the source explains the subtle part: the row must be added *inside* the savepoint. Added outside, the object stays pending in the session after the failure and is inserted again at commit, crashing it. A 20-way race test on real PostgreSQL found this; SQLite, which has a single writer, never could.

FOR UPDATE SKIP LOCKED: many dispatchers, no double dialing

Several server processes may run the dispatcher. Each selects due calls in the database:

```
due = s.scalars(select(CallAttempt)
                .where(CallAttempt.status == "scheduled", CallAttempt.scheduled_at <= now,
CallAttempt.channel == "twilio")
                .order_by(CallAttempt.scheduled_at).limit(limit)
                .with_for_update(skip_locked=True)).all()
```

`FOR UPDATE` locks the selected rows; `SKIP LOCKED` tells other workers to *skip* rows already locked instead of waiting. Two workers therefore take different calls, never the same one. This turns the table into a safe work queue with no extra infrastructure.

Commit before you dial

The order of operations in `dispatch_once` matters:

```
transition(s, call, "dialing", "dispatcher")   # committed BEFORE dialing: no other worker can
take it now
claimed.append(call.id)
```

```
...                                          # the with-block ends: the transaction commits
for cid in claimed:
    results.append({"call_id": cid, "twilio_sid": dialer(cid)})   # only now is Twilio asked to
ring
```

If the status change were made after the dial, a crash in between would leave a call that rang but was still `scheduled`, to be dialed again. Committed first, a crash leaves a `dialing` row that the **reaper** later fails (after three minutes) so the outcome engine can retry it. The cost is the opposite risk, a call marked `dialing` that never rang; that is the safer of the two failures.

The two-phase outcome

`outcome.apply` decides what happens after a call, and may ask a model. Model calls are slow and the model's tools open their own database sessions, so the function is split:

1. **Phase 1, read only.** Open a session, return the stored outcome if one exists (idempotency), classify with the rules, and gather the facts for the post-call agent. Close the session.
2. **The model runs** (outside any open transaction, so its tools can write freely).
3. **Phase 2, write.** Open a session, check again whether another worker decided while we were thinking, record the outcome, and schedule the follow-up.

Holding a transaction open across a slow model call would block other writers (and with SQLite would deadlock outright). Phase 2 also re-checks `call.outcome`, so two racing workers decide once.

Keeping a promise made live

When the agent books a call-back during the call (a real `call_attempts` row with key `<occurrence>:cb<n>`), the post-call engine must not add a second follow-up. Phase 2 looks for a scheduled attempt whose `prev_attempt_id` is this call, records `followup.already_scheduled`, and sets its own follow-up to nothing. It also withholds "plan tomorrow" until that call-back's own outcome is decided, because the occurrence is not finished.

Detecting a cut-off

A call that dropped mid-sentence needs an immediate ring-back, but "the call ended" does not say whether the parent hung up or the line failed. The evidence is gathered in layers. The **bridge** records whether a speech segment was still open or loud audio had arrived in the last 1.2 seconds. The **outcome engine** adds what the transcript shows (a question left unanswered, a last line ending in "and" or "the", under 20 seconds connected with no speech) and a goodbye detector. When those conflict, a small model decides, and its default is to ring back, the safer mistake. A hard rule in the post-call agent's guard stops a model from downgrading a cut-off when the parent was provably mid-speech.

Loops and limits

Several small caps are guarantees against runaway behaviour: at most four model steps per turn, at most 24 parent turns per call, a refusal to say the *same line* a third time (the call ends politely), three silent turns end the call (and still leave a record), and at most two attempts per reminder. Each exists because a model, left alone, will occasionally repeat itself indefinitely.

Time-of-day determinism

A quieter bug: tests that booked "a call-back in 30 minutes" began failing after 21:00 local time, because quiet hours (21:30 to 07:00) moved the booking to the morning. Real behaviour was correct; the tests depended on the wall clock. The fix was to move the quiet-hours window out of the way in tests that are not about the night. The same effect showed up in the tool-selection evaluation, which now sets its own window, and as a real product gap: when quiet hours pushed a time past the next reminder, the tool blamed "the next reminder" instead of the night. It now returns a `quiet_hours` reason with the latest time that would work.

CHAPTER 21

Testing Behaviour That Varies

A model's output differs from run to run, so a single passing run proves little and a single failing run may be bad luck. This chapter describes the layers of testing and the techniques that make them trustworthy.

Layer 1: a stand-in model

Most tests replace the model with a small class that returns scripted JSON. This tests everything *around* the model (memory, the tool loop, refusals, guarantees) exactly and fast.

```
class FakeLLM:
    def __init__(self, *replies):
        self.replies, self.calls, self.systems = list(replies), [], []

    def chat_json(self, system, messages, max_tokens=None):
        self.calls.append(copy.deepcopy(messages))      # what the model was shown, each time
        self.systems.append(system)
        return self.replies.pop(0)
```

Because the stand-in records every request, a test can assert that the agent really had memory: after two parent turns, the second call's message list must contain the first parent line and the first agent line. Another test scripts a refused booking followed by a counter-offer and checks that nothing was booked until the parent agreed.

The graph accepts the model as an argument (`CallSession(..., llm=fake)`), so no real client is needed. That is the benefit of passing dependencies in rather than creating them inside.

Layer 2: simulated parents and a judge

A language model plays an elderly person using a *persona card*: a profile, traits, and a hidden truth about their day ("you have not eaten, you have no appetite since your husband passed"). The persona sees only what a real parent would hear. A separate, stronger model, the **judge**, grades the transcript against a rubric, and must **cite the**

turn numbers behind each verdict, which code verifies exist. Notes written by a model are subordinate to deterministic checks; the order of authority is: deterministic checks, then notes, then a human.

Lessons from running this:

- A judge needs to know what the system *really did*. Early runs marked a genuine call-back booking as an "invention" because the judge saw only the words. The judge's brief now includes database facts (what was booked, logged and alerted).
- Exact-match expectations written for a scripted agent ("exactly these alerts") flagged harmless differences in a flexible agent. Expectations were relaxed only where the old test over-specified, never to hide a real fault.
- Judges disagree with themselves a little. Treat a pass rate as a signal, not a measurement, and read the failing transcripts.

Layer 3: the tool-selection test

The most useful test for the conversational agent asks a narrow question: **given this situation, does the agent call the right tools?** `evals/tool_use.py` holds 58 cases. Each has a first parent line and a specification:

```
dict(id="med_not_yet_books_callback", routine="metformin_reminder", turns=["Not yet, I will take
it soon."],
     must=[_arg("log_medicine", status="not_taken"), _arg("book_callback")])

dict(id="scope_story", routine="lunch_checkin", fixed=True, turns=["Tell me a long story."],
     forbid=["log_meal", "book_callback", "raise_alert"], forbid_say=[r"once upon|long ago"],
     ends_with_question=True, no_family_promise=True)
```

`must` lists tools that must succeed (with argument checks); `forbid` lists tools that must never even be *attempted*; `says` and `forbid_say` are regular expressions on the spoken text; `ends_with_question` checks the agent returned to its question. The test reads what the agent *did* from the audit events and the database, not from what it *said*, which matters because a model can say "I have booked it" without having done so.

Two design points make the results meaningful:

- **Follow-ups.** After the scripted first line, a cooperative simulated parent answers the agent's questions, because an agent that asks "What did you have?" and gets silence would be judged unfairly. Negative cases (a story request, an injection

attempt, an emergency) are marked `fixed`, single-turn, so no invented follow-up answers (such as the parent then saying "I had lunch") can make a forbidden call look correct.
- **Repeated runs.** Each case runs several times, since one run cannot separate a flaw from bad luck. Weak spots were re-run six times per model.

The same test measures latency and tokens for each model, which is how the cheap and the stronger model were compared on identical cases.

Layer 4: environment determinism

Tests must not depend on the clock, the machine or the network. Specific measures:

- The quiet-hours window is moved out of the way unless a test is about the night (Chapter 20).
- Each test gets its own database. SQLite uses a file, not `:memory:`, because an in-memory database shares one connection across threads, which would let the server thread and the test thread corrupt each other's transactions.
- Tests that use real concurrency run only on PostgreSQL, selected by `TEST_DATABASE_URL`.
- The evaluation processes each have their own database because SQLAlchemy's engine is a per-process global.

Layer 5: real services, measured

A few checks run against the real world and produce numbers rather than pass or fail: `carecall latency` (speech and model stage times), `model-bench` (speed and accuracy of candidate models), `live-scheduling` (a real model and a real PostgreSQL), and the text-to-speech comparison (time to first audio and a control comparing whole-clip with streamed audio). These are not in the 230-test suite because they cost money and vary.

What to take from this

A test for model behaviour is only as good as the question it asks. "Does the conversation sound good?" is hard to test; "does it call `book_callback` with `in_minutes` between 15 and 25 when asked for 20 minutes?" is easy and finds real

bugs. Each bug found by a persona or a live run became one permanent regression test.

CHAPTER 22

Tracing One Call Across Threads

A single phone call runs across several threads, several HTTP requests and, at the end, a model call that happens after the audio has closed. Tracing it as one story takes some care. This chapter explains the techniques in `tracing.py` and the second source of truth, the events table.

One trace per call, created in advance

Langfuse groups work into a **trace**. The natural way to create one is to open a span and let everything inside belong to it. That does not work here: each turn runs in a different worker thread, and the post-call work runs later still, so no single span stays open for the whole call.

The solution is to create the **trace id up front**, when the session is created (`tracing.new_trace_id()`), and attach every piece of work to that id explicitly:

```
@contextlib.contextmanager
def call_span(trace_id, name, call_id, parent_id, routine_key, tags=None, input=None, ...):
    with propagate_attributes(session_id=session_id or f"call-{call_id}",
                              user_id=user_id or f"parent-{parent_id}",
                              tags=(tags or []) + [routine_key], version=s.prompt_version,
trace_name=..., metadata=...):
        with _client.start_as_current_observation(trace_context={"trace_id": trace_id},
name=name, as_type="span", input=input) as span:
            yield span
```

`trace_context={"trace_id": trace_id}` is what ties each span to the call's trace no matter which thread created it. `propagate_attributes` stamps the session id, the Langfuse user, the tags and the prompt version on everything created inside the block, so each turn, node, tool and model call carries them.

A decorator that costs nothing when tracing is off

Nodes, tools and model calls are traced with a decorator:

```
def traced(name, as_type="span"):
    def deco(fn):
        wrapped = None
        @functools.wraps(fn)
        def inner(*args, **kwargs):
            nonlocal wrapped
            if _client is None:                    # tracing off: call the function directly
                return fn(*args, **kwargs)
            if wrapped is None:                    # first traced call: wrap it with Langfuse's
observer, once
                from langfuse import observe
                wrapped = observe(name=name, as_type=as_type)(fn)
            return wrapped(*args, **kwargs)
        return inner
    return deco
```

Two things are worth noticing. The check `_client is None` happens at *call* time, not at decoration time, so the same decorated function works whether or not tracing is started later (it is started when the server boots, after modules are imported). And the Langfuse import is deferred until it is needed, so the package imports fine without Langfuse installed or configured. A failure to initialise tracing only logs a warning; it can never stop a call.

Users, sessions and the admin view

Each real parent is a separate Langfuse **user** named `parent-<id>-<name>`, so an operator can open the Users tab and see only that person's calls. Each call is a **session** named `call-<id>`. Evaluation runs use `eval:<scenario>` as the user, so tests never mix with real parents. The admin endpoints return these names (`langfuse_user`, `langfuse_session`) next to each call, so an operator can jump from a call in the database to its trace.

What a trace contains

A turn's trace is a tree: the turn span; inside it the graph nodes (`safety`, `agent`, `tools`, `finish`); inside those the model call (a *generation*: the prompt, the reply and the model) and each tool (`db_tool:book_callback`). After hang-up, a `post_call` span holds the summary, the outcome decision and, if used, the post-call agent's loop. Scores (for example `eval_passed`) attach to the trace id.

Deployed Langfuse v4 runs in an "events only" mode in which the older trace-listing endpoints are disabled, so queries use the observations endpoint instead. That affects only anyone scripting against the API.

The events table: a second source of truth

Tracing is optional and lives outside the database. The `events` table is the record the application itself relies on. Every state change, tool call, model step, outcome decision and follow-up writes an event with a type and a JSON payload (`state.changed`, `db_tool.book_callback`, `converse.step`, `converse.tools`, `converse.loop_guard`, `outcome.decided`, `followup.already_scheduled`, `call.finished`, and many more). A call can be reconstructed from these events alone, even if Langfuse was down. Several parts of the system read them: the outcome engine reads `call.finished`; a ring-back reads where the previous call stopped; the evaluations read `converse.step` to see which tools the agent really called.

Counting model use

The model client keeps its own counters: calls, seconds, the duration of each call, prompt, completion and reasoning tokens. The evaluation harness reads them to report median and p90 latency and tokens per run, which is how the two models were priced and compared. It is a small piece of instrumentation that answers "what does a call cost?" without an external tool.

PART V

Choices and Alternatives

The other stacks you could have used, including live speech-to-speech models, and a checklist for your next application.

CHAPTER 23

Alternative Stacks and Their Tradeoffs

Every technology in this book was chosen against alternatives. This chapter lays those alternatives out, layer by layer, so that on your next application you can make the same kind of decision deliberately instead of copying a stack. The comparisons are qualitative: no prices or benchmark numbers are quoted, because they change quickly and depend on your workload. Product names are examples of a category, not endorsements, and anything you adopt should be checked against its current documentation.

The five forces behind every choice

Almost every decision below came down to the same five forces. Name them before you pick a tool.

1. **Latency.** A phone conversation feels broken if the agent pauses for more than a second or two. This pushed us toward a small, non-thinking model for the live path and away from anything that adds a network hop per turn.
2. **Control versus speed to build.** A managed platform gets you a demo in a day but hides the places where guarantees must be enforced. CareCall needs a guardrail that *cannot* be talked out of, so the control had to live in our code.
3. **Data sensitivity.** The product handles the health and daily routine of elderly people. That favoured self-hosted tracing and a database we own.
4. **Scale and operations.** A handful of parents needs very different infrastructure from a million. We deliberately chose tools with few moving parts.
5. **Team skills and learning.** This was a learning project, so choosing tools that expose the underlying ideas (WebSockets, state machines, SQL locking) had value in itself.

A stack that is right for one weighting of these forces is wrong for another. The tables that follow say which force each choice was optimising.

What we used, in one table

Layer	We used	Main advantage here	Main cost
Phone network	Twilio Voice and Media Streams	Raw audio over a WebSocket, so we keep control of speech and logic; mature documentation	Narrowband 8 kHz audio, per-minute cost, vendor dependence for the number
Speech	Gnani Prisma (STT) and Timbre (TTS)	Built for Indian languages and code-mixed speech such as Hinglish	Smaller ecosystem than the global providers; one more vendor key
Language model	OpenAI-compatible API through OpenRouter; small non-thinking Qwen live, stronger model off-line	One interface and one key; swap a model by changing a string; fast live replies	A middleman in the path; hosted models mean data leaves the machine
Conversation	LangGraph running a model-driven loop with tools	Explicit nodes, routes and saved state; a clean home for the agent loop; per-node tracing	An extra dependency; the loop could also be plain Python
Web and concurrency	FastAPI on ASGI with asyncio	Native WebSockets and async; many calls on one thread	Blocking code must be pushed to threads or it freezes every call
Database	PostgreSQL 17 with SQLAlchemy	Row locks and SKIP LOCKED make it a safe queue; one system for state, queue and audit log	A database queue has a throughput ceiling; no migrations yet
Tracing	Langfuse, self-hosted	Health data stays local; LLM-aware views	Six containers to run and upgrade
Evals	Our own persona harness, judges and pytest	Checks that know the domain (no medical advice, quotes must be verbatim)	We maintain it ourselves

Layer 1: telephony and the voice transport

The decision here is who carries the audio and in what protocol.

Option	Typical advantage	Typical cost	Pick it when
Twilio Media Streams (used)	Mature, well documented, raw audio over WebSocket, strong status webhooks	Higher per-minute cost; 8 kHz mu-law audio limits speech quality	You want control of the speech stack and a dependable path to the phone network

Option	Typical advantage	Typical cost	Pick it when
Other cloud telephony (Telnyx, Vonage, Plivo)	Similar model, sometimes cheaper or with different coverage	Smaller communities; feature gaps differ	Price or number availability decides it
Local Indian providers (Exotel, Ozonetel and similar)	Local numbers, regulatory fit for calling in India	Different APIs; fewer English-language examples	You call at scale inside India and compliance drives the choice
WebRTC stacks (LiveKit, Daily with Pipecat)	Wideband audio, built-in interruption handling, low latency between browsers and apps	You operate more infrastructure; a SIP bridge is needed to reach ordinary phones	The user is in an app or browser, or you want the best audio quality
Managed voice-agent platforms (Vapi, Retell, Gnani Agent Builder)	Fastest route to a working agent, hosting included	The decision loop lives inside the vendor; harder to enforce code-level guarantees or reach your database	A prototype, or a use case where the vendor's tools already cover your needs

Why we chose as we did. Media Streams let us put our own guardrails, database tools and tests around the conversation. A managed platform would have been quicker to start, but Chapters 19 and 20 depend on code owning the guarantees (the tool rules, the attempt cap, idempotent scheduling). The price is lower audio quality and a vendor dependency on the phone number.

Transport sub-choice. A WebSocket is a long-lived two-way channel and fits real-time audio. Server-Sent Events are simpler for one-way streams such as model tokens. gRPC streaming is efficient between your own services but is not what telephony vendors offer. HTTP request and response is right for webhooks. We used each where it fits and measured SSE for text-to-speech (first audio in 0.06 seconds against 0.56 seconds for the whole file) without yet adopting it live.

Layer 2: speech in and out

Option	Advantage	Cost	Pick it when
Gnani STT and TTS (used)	Indian languages and code-mixed speech	Narrower ecosystem and fewer third-party examples	Your callers speak Indian languages or mix them with English
Global speech APIs (Deepgram, AssemblyAI, Google, Azure)	Mature streaming, broad language lists, good tooling	Accuracy on code-mixed regional speech varies; test with your own audio	English or major languages dominate

Option	Advantage	Cost	Pick it when
Open-source models you host (Whisper variants and others)	Data stays with you; no per-minute fee	You operate GPUs and handle streaming yourself; latency needs work	Privacy or cost at volume outweighs the operational load
Expressive TTS specialists (ElevenLabs, Cartesia)	Very natural voices, low time-to-first-audio	Language coverage and phone-quality tuning differ	Voice quality is the product
Speech-to-speech models (OpenAI Realtime, Gemini Live)	One model hears and speaks; lowest conversational delay, natural interruption	Harder to audit and to enforce deterministic rules; the model, not your code, shapes the wording	Open-ended conversation where exact wording does not matter

The tradeoff that matters. A cascaded pipeline (STT then LLM then TTS) is slower than speech-to-speech, but every stage produces text you can log, check and test. CareCall needs exact reminder wording, verbatim evidence for each logged fact and a dependable emergency path, so it accepts the extra delay and works to reduce it (a small model, streaming TTS as the next step).

Live speech-to-speech models

A newer option replaces the whole chain (speech-to-text, model, text-to-speech) with **one model that listens and speaks**. It handles interruption natively and removes the delay of three network hops. The facts below were checked against each provider's own documentation in October 2026; pricing and language lists change, so verify them before deciding.

Option	What was confirmed	Fit for CareCall
OpenAI Realtime API (`gpt-realtime-2.1` and a cheaper mini)	Speech-to-speech over WebSocket, WebRTC and incoming SIP. Priced per token (audio $32 and $64 per million tokens in and out). Outbound SIP calling is **not** offered on this API	Incoming calls only, which does not suit a service that places the calls
OpenAI GPT-Live (`gpt-live-1`)	Full duplex. Outbound and inbound SIP, but outbound must be enabled for the organisation. $0.05 per minute plus the backend model. Tools are *delegated*: your backend runs them while the voice model keeps talking. Documentation lists Hindi (one voice) among seven languages	Good fit for the call pattern and the tool design; language coverage for regional Indian speech is the question

Option	What was confirmed	Fit for CareCall
Google Gemini Live API	Official docs list 99 languages including Hindi, Tamil, Telugu, Kannada, Marathi, Bengali, Malayalam and Gujarati. Function calling, voice activity detection and interruption, WebSocket with 16 kHz audio in and 24 kHz out. The model picks the language automatically (it cannot be forced, only steered by instructions). Audio-only sessions are limited to 15 minutes. No native phone support documented	The strongest candidate on language coverage and price. Its WebSocket shape resembles our existing bridge, which already converts phone audio
Amazon Nova 2 Sonic	Bidirectional streaming on Bedrock. Hindi supported, with six other languages. Phone integration not confirmed	Reasonable if the stack is already on AWS; Hindi only among Indian languages
Ultravox, ElevenLabs Agents, Deepgram Voice Agent, Retell	Vendor claims only. Several are platforms or pipelines rather than single models	Unverified for Indian languages

What the choice costs in control. With a single model the voice is generated directly, so the checks CareCall runs before speech (the word cap, the leak filter, the repeated-line guard, the emergency check ahead of the model) become *after-the-fact* checks on the transcript. A safety rule that was a gate becomes an alarm. Whether that is acceptable depends on how much each rule matters. Tool rules (quiet hours, evidence, the attempt cap) are unaffected, because they live in the tools.

What to test before choosing. The same scenarios on each option with a real phone: the delay to first audio, how it handles accented and elderly speech, whether a regional language is answered in the right language, and whether tool calls are correct. OpenRouter, which the project uses for text models, offers no live speech-to-speech model: its Gemini entries are text-output only, and its `gpt-audio` models take a clip of audio and return audio per request, which is neither full duplex nor interruptible.

Indian-language providers (a cascade like today's)

Keeping the chain but changing its parts is the lower-risk path. **Sarvam** documents speech-to-text in 23 languages (22 Indian plus English) with a code-mixed mode and streaming, text-to-speech in 11, a managed voice-agent platform in 11 Indian languages from ₹3 per minute, and integrations with Twilio, Exotel, LiveKit and Pipecat. Because CareCall speaks to its speech services through two small interfaces

(`STTProvider` and `TTSProvider`), swapping in another provider means writing an adapter, not touching the agent, the guardrails or the tools. The cost is the same latency profile as today.

Layer 3: language models and how you call them

Where the model runs.

Option	Advantage	Cost
OpenAI-compatible gateway (OpenRouter), used	One API and key for many models; trivially swappable; good for benchmarking	An extra hop; availability and privacy terms of both gateway and provider
Direct provider SDKs	Newest features first; fewer parties	Each vendor needs its own client code
Cloud platforms (Bedrock, Vertex AI, Azure OpenAI)	Enterprise contracts, regional hosting, existing security reviews	More configuration; model catalogue differs
Self-hosted (vLLM, Ollama, llama.cpp)	Full data control; predictable cost at volume	You run, scale and secure inference yourself
A library gateway (LiteLLM and similar)	Unified interface you run yourself	One more component to operate

Which model. A small non-thinking model answers in about a second, which is what a live call needs; a stronger model is used only where latency does not matter (judging, the post-call decision). Choosing one big model for everything is simpler but makes every turn slower and costlier. Measure with your own prompts before deciding; our benchmark showed the cheaper model understood elderly speech well enough once the code around it validated its output.

How you get structured output. There are four common ways.

Method	Advantage	Cost
Ask for JSON and validate it in code (used)	Works with any model; one round trip; code decides what to trust	Occasional malformed output needs a retry or fallback
Native function or tool calling	Standard, and the model picks the tool	Weaker on small models; extra round trip when tool output goes back to the model

Method	Advantage	Cost
Provider structured outputs or constrained decoding (including libraries such as Instructor and Outlines)	Output always matches the schema	Not available on every model; the shape is guaranteed but not the truth of the content
Free text with parsing	Simplest	Fragile

We used JSON plus validation because it works with any model in one round trip, and because the *quote check* (the evidence must appear in what the parent said) catches errors that a schema cannot.

Layer 4: orchestrating the conversation

Option	Advantage	Cost	Pick it when
LangGraph (used)	Explicit nodes, conditional routing and saved state per thread; each node shows up in traces	Another dependency and concepts to learn	The flow branches, state matters and you want checkpointing or human-in-the-loop later
A plain Python state machine	Fewest moving parts; easiest to reason about	You write persistence, tracing hooks and routing yourself	A small, stable flow; honestly sufficient for CareCall's current size
Other agent frameworks (LlamaIndex Workflows, CrewAI, AutoGen, OpenAI Agents SDK, Pydantic AI)	Strong built-in patterns for tool use and multi-agent set-ups	Often let the model run the whole flow; here the model chooses words and tools but code enforces the rules	The model should plan and call tools freely
Durable workflow engines (Temporal, Inngest, Hatchet)	Survive crashes and restarts; excellent for long-running processes	Heavy for a short conversation; more infrastructure	Multi-day processes with retries and timers
Classic dialogue managers (Rasa, Dialogflow CX)	Intent and slot-filling built for scripted dialogue	Less natural with mixed speech; weaker with large language models	A closed menu of intents and no generative step

The honest verdict. LangGraph was chosen partly to learn it and partly for explicit routing, saved state and per-node tracing. Its job here is the agent loop (ask the model, run its tools, show it the results, repeat), which could also be written as a plain function; the graph earns its keep as soon as you add persistence across restarts, approvals or parallel work. The project went through three designs. A

scripted question flow, where a model filled a form for each answer and code decided what to say, was predictable but could not hold a conversation: a parent negotiating a call-back time was beyond it. A **model that simply talked**, with no tools, sounded natural but promised things nothing fulfilled and reached only 14 to 20 of 25 test personas. The current design is a **model-driven loop with tools**: the model chooses the words and which tools to call, and code enforces the rules inside the tools, so a promise is spoken only after a tool has confirmed it.

Layer 3: language models and how you call them

Where the model runs.

Option	Advantage	Cost
OpenAI-compatible gateway (OpenRouter), used	One API and key for many models; trivially swappable; good for benchmarking	An extra hop; availability and privacy terms of both gateway and provider
Direct provider SDKs	Newest features first; fewer parties	Each vendor needs its own client code
Cloud platforms (Bedrock, Vertex AI, Azure OpenAI)	Enterprise contracts, regional hosting, existing security reviews	More configuration; model catalogue differs
Self-hosted (vLLM, Ollama, llama.cpp)	Full data control; predictable cost at volume	You run, scale and secure inference yourself
A library gateway (LiteLLM and similar)	Unified interface you run yourself	One more component to operate

Which model. A small non-thinking model answers in about a second, which is what a live call needs; a stronger model is used only where latency does not matter (judging, the post-call decision). Choosing one big model for everything is simpler but makes every turn slower and costlier. Measure with your own prompts before deciding; our benchmark showed the cheaper model understood elderly speech well enough once the code around it validated its output.

How you get structured output. There are four common ways.

Method	Advantage	Cost
Ask for JSON and validate it in code (used)	Works with any model; one round trip; code decides what to trust	Occasional malformed output needs a retry or fallback

Method	Advantage	Cost
Native function or tool calling	Standard, and the model picks the tool	Weaker on small models; extra round trip when tool output goes back to the model
Provider structured outputs or constrained decoding (including libraries such as Instructor and Outlines)	Output always matches the schema	Not available on every model; the shape is guaranteed but not the truth of the content
Free text with parsing	Simplest	Fragile

We used JSON plus validation because it works with any model in one round trip, and because the *quote check* (the evidence must appear in what the parent said) catches errors that a schema cannot.

Layer 4: orchestrating the conversation

Option	Advantage	Cost	Pick it when
LangGraph (used)	Explicit nodes, conditional routing and saved state per thread; each node shows up in traces	Another dependency and concepts to learn	The flow branches, state matters and you want checkpointing or human-in-the-loop later
A plain Python state machine	Fewest moving parts; easiest to reason about	You write persistence, tracing hooks and routing yourself	A small, stable flow; honestly sufficient for CareCall's current size
Other agent frameworks (LlamaIndex Workflows, CrewAI, AutoGen, OpenAI Agents SDK, Pydantic AI)	Strong built-in patterns for tool use and multi-agent set-ups	Often let the model run the whole flow; here the model chooses words and tools but code enforces the rules	The model should plan and call tools freely
Durable workflow engines (Temporal, Inngest, Hatchet)	Survive crashes and restarts; excellent for long-running processes	Heavy for a short conversation; more infrastructure	Multi-day processes with retries and timers
Classic dialogue managers (Rasa, Dialogflow CX)	Intent and slot-filling built for scripted dialogue	Less natural with mixed speech; weaker with large language models	A closed menu of intents and no generative step

The honest verdict. LangGraph was chosen partly to learn it and partly for explicit routing and per-node tracing. A plain state machine would also have met today's needs. The graph earns its keep as soon as you add persistence across restarts,

branching approvals or parallel work. We also built a model-written agent for comparison; it sounded natural but skipped steps and reached 14 to 20 of 25 test personas, which is why the graph, not free generation, runs production.

Layer 5: web framework and concurrency

Option	Advantage	Cost	Pick it when
FastAPI on asyncio (used)	Native WebSocket and async, Pydantic validation, auto docs	Blocking calls freeze the loop unless moved to threads	Python team, many concurrent waiting tasks
Flask or Django (synchronous)	Huge ecosystems; simple mental model	One thread per connection; WebSockets need add-ons such as Channels	Ordinary request and response apps
Node.js (Fastify, Express)	Excellent event loop; strong real-time ecosystem	Less convenient for ML and data code	The team is TypeScript-first
Go	Cheap concurrency and a single binary	Fewer AI libraries; more code for the model side	Very high connection counts or strict latency
Elixir on the BEAM	Built for millions of lightweight, fault-isolated connections; strong in telecom	Smaller AI ecosystem and talent pool	Telephony at very large scale

Why asyncio. A phone call is mostly waiting: for speech, for the model, for audio. One thread can serve many such calls if nothing blocks it. The cost is discipline: every database call and every graph turn is sent to a worker thread so the audio of other calls never stalls.

Layer 6: data, queues and background work

Option	Advantage	Cost	Pick it when
PostgreSQL as store and queue (used)	One system; the queue is transactional with the state; SKIP LOCKED gives safe multi-worker dispatch	Throughput ceiling far below dedicated brokers; polling	Modest to high thousands of jobs a minute and the need for strong consistency
SQLite	Zero setup	One writer; no row locks	Tests and prototypes
Redis-backed queues (Celery, RQ, Dramatiq)	Mature retry and scheduling patterns; very fast	A second system whose state can drift from the database	Many short background tasks, tolerant of eventual consistency

Option	Advantage	Cost	Pick it when
Cloud queues (SQS) or log systems (Kafka, Redis Streams)	Scale and durability beyond a database	Operational weight; ordering and exactly-once are hard	Very high volume or many consumers
Document stores (MongoDB, DynamoDB)	Flexible schema; scale-out	Weaker relational guarantees; unique-key and transaction patterns differ	Large, loosely structured data with simple access patterns
Durable workflow engines (Temporal)	Retries, timers and state handled for you	A new platform to learn and run	Complex, long-lived flows

Why a database queue. A call's status, its idempotency key and its place in the queue are the same row. Claiming it, marking it dialing and committing is one transaction, which removes a whole class of duplicate-dial bugs. The tradeoff is scale: this design is excellent for thousands of calls an hour and the wrong tool for millions.

ORM and migrations. SQLAlchemy 2 was chosen for portability between Postgres and SQLite tests and for explicit transactions and SAVEPOINTs. Raw SQL with a thin driver is faster to reason about for a few queries; SQLModel and Django's ORM trade flexibility for convenience. Alembic is the planned migration tool and is the main missing piece before production.

Layer 7: tracing and evaluation

Option	Advantage	Cost
Langfuse, self-hosted (used)	Open source, LLM-aware views, data stays on your machines	Six containers to run and upgrade
Hosted LLM observability (LangSmith, Braintrust, Helicone, Weave)	No infrastructure; polished UI and eval tooling	Sensitive transcripts leave your control; per-seat or usage cost
Arize Phoenix	Open source and light to start	Smaller feature set for prompt management
Plain OpenTelemetry with a general backend	One tracing system for everything	You build the LLM-specific views

For evaluation, off-the-shelf frameworks (promptfoo, DeepEval, Ragas, Inspect) speed up common patterns such as retrieval scoring or prompt comparison. We wrote our own harness because the important checks here are specific to the domain: a log is only valid if the parent's quote supports it, the agent must never give medical

advice, and persona behaviour must match a hidden ground truth. The cost is that we maintain the harness; the benefit is that each failure becomes a precise regression test.

Layer 8: packaging and deployment

CareCall runs with Docker Compose for the databases and a plain process for the server. That is right for development and a first pilot. Alternatives matter once you deploy:

Option	Fits	Watch out for
A virtual machine or Compose on one host	Pilot, small scale	You manage uptime, backups and upgrades
Container platforms (ECS, Cloud Run, Fly, Render)	Managed scaling and deploys	Long-lived WebSockets and timeouts need checking; sticky behaviour for in-memory call state
Kubernetes	Large teams and many services	A lot of operational overhead for one service
Serverless functions	Short stateless webhooks	Poor fit for a live audio WebSocket that stays open

Note one design consequence: the per-call conversation state sits in process memory. That is fine while each call stays on one server, and it means a load balancer must keep a call's WebSocket on the same instance.

Components CareCall did not need, but your next application might

- **Retrieval and vector search.** If the agent must answer from documents (policies, manuals, medical leaflets), add a vector index. Postgres with the pgvector extension keeps one database; dedicated stores (Qdrant, Weaviate, Pinecone, Chroma) scale larger or offer richer filtering. Judge retrieval quality with its own metrics, not only the final answer.
- **Long-term memory.** Summaries of past calls could inform the next one. Start with plain tables and a summary field before adding a memory framework.
- **Caching and rate limiting.** Redis in front of model calls reduces cost for repeated prompts; add it when you can measure a hit rate.
- **Fine-tuning.** Consider it only after prompts, validation and evals plateau; a fine-tuned small model can cut latency and cost on a narrow task.

- **MCP (Model Context Protocol).** A standard way to expose tools to models across applications. We used an in-process registry because everything runs in one codebase; MCP becomes attractive when several applications or assistants should share the same tools. Whatever the transport, the same rules apply: pin the user identity server-side, check arguments, log every call.
- **Content moderation services.** A provider moderation API can supplement, never replace, deterministic guardrails.

Four reference stacks you can adapt

Application	A sensible starting stack	Why
Voice agent (this book)	Telephony vendor with raw audio, cascaded STT, small LLM, TTS, a graph or state machine, Postgres, Langfuse	Control and testability; text at every stage
Chat or RAG assistant	Web framework, a hosted LLM, pgvector or a vector store, streaming responses over SSE, tracing with an LLM tool	Streaming text and good retrieval matter more than telephony
Document processing pipeline	Object storage, a queue, batch LLM calls with structured outputs, a validation step, human review UI, Postgres	Throughput and correctness; no live latency constraint, so use the strongest model
Long-running agent workflow	A durable workflow engine for state and retries, tool calling with strict permissions, audit log, evals on trajectories	Crashes and timers dominate; each step must be resumable and auditable

A decision checklist for your next application

1. **What is the latency budget?** Under two seconds per turn forces small models, streaming and few network hops; minutes or hours lets you use the strongest model and simple batch design.
2. **What must be guaranteed?** List the rules that can never be broken (no advice, no duplicates, always escalate). Put them in code, and test them first.
3. **Who owns the data?** Health, finance and children's data usually push toward self-hosted tracing, regional hosting and minimal prompts.
4. **How much traffic, and how spiky?** Pick the simplest infrastructure that survives ten times today's load, not one hundred times.
5. **Does the model plan, or does your code plan?** Free planning is flexible but unpredictable; a fixed flow with model interpretation is predictable and testable. Mix them by putting autonomy where mistakes are cheap.

6. **Can you observe and replay a failure?** If you cannot reconstruct what the system knew and decided, you cannot fix it. Choose tracing and an append-only event log early.
7. **How will you test non-deterministic behaviour?** Combine deterministic checks, simulated users, a judge with cited evidence, and one regression test per bug found.
8. **What is the cost of being wrong, and who pays?** The answer decides how much confirmation, human review and conservative defaulting the product needs (CareCall's default is to ring back, the safer mistake).

What we would change first if requirements grew

- Extend streaming to the model (speaking sentence by sentence). Streaming text-to-speech is already in; the model's reply is still generated in full before speech starts.
- Add **Alembic migrations** and a managed Postgres with backups.
- Move call state from process memory to a store that survives a restart, or add sticky routing, before running more than one server for live calls.
- Add an **onboarding** interface and caregiver authentication, and replace the console notifier with SMS or push.
- Consider **WebRTC or a speech-to-speech model** for a future app-based product, accepting less deterministic wording in return for more natural interaction, but only for conversations where that wording is not safety-critical.

None of these change the central idea. Whatever tools you choose, keep the model's job narrow, put the guarantees in code, record everything as events, and test with simulated users before real ones.

PART VI

Operating It

Commands, settings, measured results, and an honest list of what is still missing.

CHAPTER 24

Running It, Measured Results and What Is Missing

First-time setup

1. Activate the virtual environment: `. .venv/Scripts/activate` (Git Bash on Windows).
2. Start the application database: `docker compose -f docker/app/docker-compose.yml up -d`. Use `127.0.0.1`, not `localhost`.
3. Optional but recommended, start Langfuse: `cd docker/langfuse && docker compose up -d`, then `carecall langfuse-sync`.
4. Fill `.env` (copy `.env.example`): `DATABASE_URL`, the Gnani speech key, the model base URL, key and names, Langfuse keys, and later the Twilio account id, token, number and `PUBLIC_BASE_URL` (an HTTPS tunnel Twilio can reach).
5. Create demo data: `carecall seed`.

Commands

Command	What it does
`carecall chat`	Talk to the agent in the terminal (text channel, nothing is dialed)
`carecall serve`	Start the FastAPI server: webhooks, the audio WebSocket, admin endpoints
`carecall plan-day [--date D]`	Create the day's calls from the stored data (safe to repeat)
`carecall schedule`	Show the calendar of calls
`carecall scheduler`	Run the dispatcher: plan, reap stale calls, dial due calls
`carecall outcome <call id>`	Show or apply the outcome decision for one call
`carecall tool-use [--runs N] [--case IDS]`	Test that the conversational agent calls the right tools; prints latency and tokens

Command	What it does
`carecall personas --mode llm --agent converse`	Run the 25 simulated parents (plus guardrails) against the conversational agent
`carecall live-scheduling`	Live test of in-call booking with simulated parents
`carecall eval`	The 17 scripted scenarios, offline
`carecall latency`, `model-bench`	Time the speech stages; compare candidate models
`carecall call`, `call-test`	Place a call through Twilio, or the simplest possible test call
`carecall gnani-*`, `twilio-*`, `voice-selftest`, `check-keys`, `trace-check`, `langfuse-sync`	Connectivity checks and the Gnani and Twilio tooling

Run the tests with `python -m pytest -q`. To include the PostgreSQL concurrency tests, set `TEST_DATABASE_URL=postgresql+psycopg://carecall:carecall@127.0.0.1:5433/carecall` first.

Settings worth knowing

Setting	Default	Meaning
`agent_mode`	converse	The conversational agent; `graph` selects the older scripted one
`tts_streaming`	true	Speak while Gnani is still generating the audio
`max_attempts_per_occurrence`	2	Calls per reminder before giving up
`cutoff_retry_delay_s`	0	Ring back at once after a cut-off
`unreachable_delays_min`, `technical_delays_min`, `hangup_retry_delay_min`	10, 5, 30	Retry delays
`postpone_live_call_s`	300	Wait when the parent is on another call
`quiet_start`, `quiet_end`	21:30, 07:00	No calls at night, parent-local
`stale_dial_s`, `stale_call_s`	180, 1500	When the reaper fails stuck calls
`scheduler_interval_s`	10	Dispatcher tick

Setting	Default	Meaning
`enable_scheduler`	false	Start the scheduler inside the server
`postcall_agent_enabled`	true	A model decides the next step after a call, guarded by rules
`admin_token`	empty	If set, `/admin/*` needs a bearer token

Measured results

Measurement	Result
Automated tests	230 passing on PostgreSQL
Tool-selection test, cheap model (Qwen Flash)	111 to 113 of 116 runs correct
Tool-selection test, stronger model (GPT-6 Luna)	114 of 116
Median turn latency (model plus tools, parent waits)	about 1.2 s cheap, 1.5 s stronger
Model cost per typical five-call check-in (estimate)	about $0.0005 cheap, $0.0015 stronger
Time to first audio, streaming speech	0.19 s, against 0.71 s for a whole clip
Persona suite with simulated parents	18 of 25 on an earlier prompt; remaining failures were mainly strict judge items

What is still missing

- **Migrations.** Tables come from `create_all`; Alembic is needed before real data.
- **Onboarding.** There is no signup or setup API or screen; data comes from `seed.py`.
- **A real outbound call.** The Twilio account details are not configured here yet, so no real call has been placed. The streaming speech and filler paths are tested against a fake bridge and the real speech service, not on a phone.
- **Streaming the model.** Only the speech streams; the model's reply is generated in full first.
- **No cutoff for very overdue calls.** After a long outage, every overdue call would ring at once, even a breakfast call at 3 pm. The dispatcher should skip or

cancel a call that is more than some minutes late and tell the family.

- **Opt-out during the call.** "Stop calling me" is acted on after the call ends.
- **Production concerns.** Beyond a shared token, authentication on the admin endpoints; secrets management; rate limits; backups; a notification channel beyond the console.
- **A model choice.** The stronger model fixes three weaknesses of the cheap one but drops a reminder request in some runs; the choice and a prompt fix for that are open.
- **Live voice.** Gemini Live, GPT-Live and an Indian-language cascade (Sarvam) have been researched but not tried.

APPENDIX A

Glossary

Term	Meaning in this project
Active-call guard	The check that stops two different calls ringing the same parent at once (`has_live_call`)
ASGI	The async interface between a Python web app and its server; what lets FastAPI serve WebSockets
asyncio	Python's single-threaded concurrency for waiting tasks, driven by an event loop
Attempt	One row in `call_attempts`: a planned or made call
Barge-in	The parent speaks over the agent; we send Twilio `clear` so our audio stops
Caregiver	The adult child who is notified and who fixes the meal times
Checkpointer	LangGraph storage that saves the state per `thread_id` between invokes
Cut-off	A call that dropped while the parent was mid-sentence or a question was unanswered
Dispatcher	The loop that claims due calls and dials them
Dry run	A tool mode that validates and describes what it would do, without writing
Event	An append-only audit row in `events`
Evidence	The parent's exact words that justify a logged fact; the tool checks they were really said
Filler	A short human stall such as "Umm... hold on a moment", spoken while a tool runs
Frame	20 ms of audio: 160 bytes of mu-law or 320 bytes of 16-bit PCM at 8 kHz
Generation	A recorded model call in Langfuse
Guardrail	A rule in code that bounds what the agent may say or do, whatever the model
Idempotency key	A unique string that makes "create this call" safe to repeat
Judge	A stronger model that grades a transcript against a rubric and must cite turns
LangGraph	The library that runs the per-call graph of nodes with saved state
Media Stream	Twilio's WebSocket carrying live call audio
mu-law	The 8-bit telephone audio encoding at 8 kHz
Occurrence key	`parent:routine:local-date:slot`, the identity of one planned call

Term	Meaning in this project
Outcome	The classification of a finished call (completed, cut_off, and so on)
Persona	A simulated parent with a profile and a hidden truth about their day
Post-call agent	The stronger model that proposes the next step after a call, clamped by rules
Reaper	The job that fails calls stuck in dialing or in progress
result_matters	A tool flag meaning the agent must see the result before it speaks
SAVEPOINT	A nested transaction that lets one failed insert be undone without losing the rest
Scope	Which caller a tool may be used by: agent, system or admin
SKIP LOCKED	A PostgreSQL option so concurrent workers skip rows already claimed
SSE	Server-Sent Events, a one-way HTTP stream; used for streaming text-to-speech
STT / TTS	Speech-to-text and text-to-speech
Tool	A registered function that reads or writes the database, with a schema and a scope
TwiML	Twilio's XML instructions for what to do with a call
Turn	One parent utterance plus the agent's reply; one `graph.invoke`
WebSocket	A long-lived two-way connection started as an HTTP upgrade